

数据表征到数据流编排的 存算协同优化

刘方鑫

上海交通大学 计算机学院

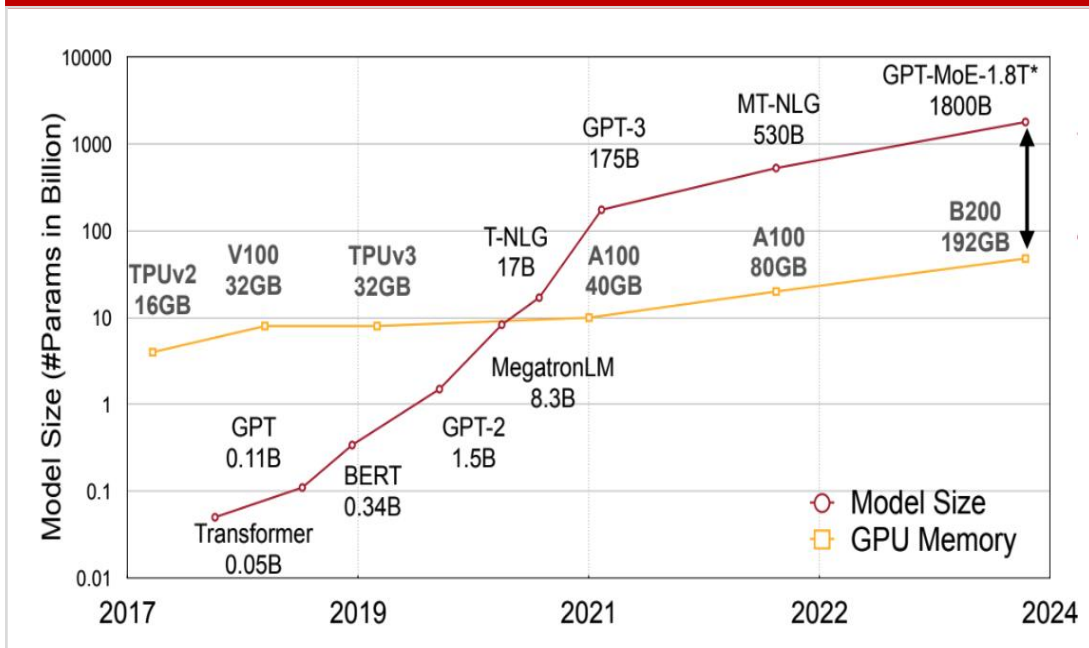
2026-6-30

饮水思源 · 爱国荣校

研究背景： AI发展引发算力/存储瓶颈

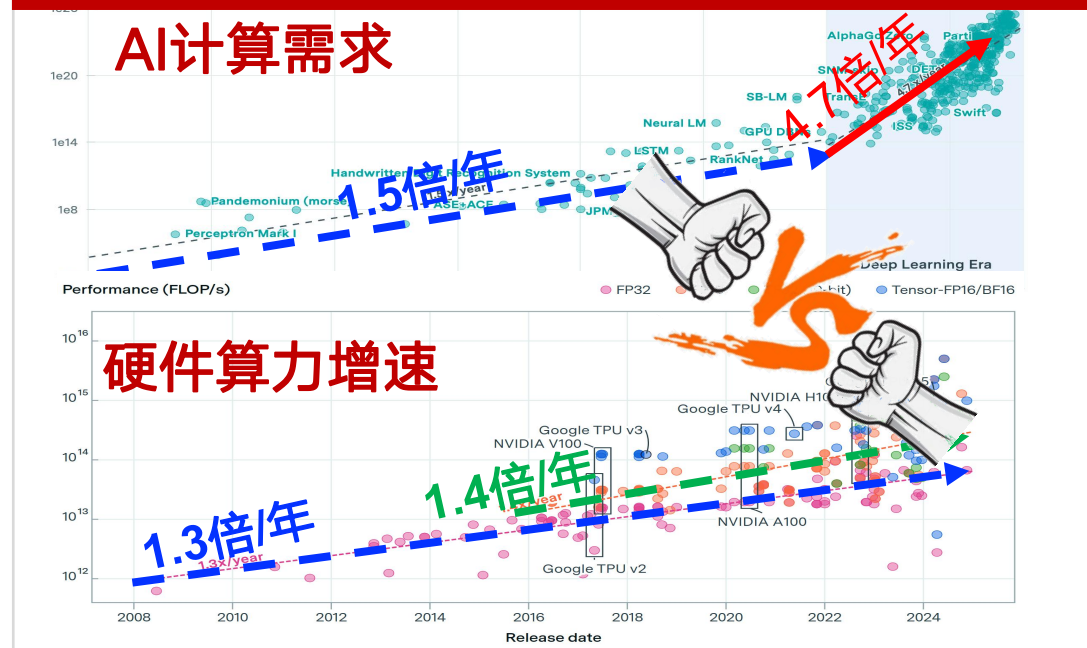
- 万亿级参数规模大模型需要百GB级存储容量，为主要成本挑战；
- 算力增速1.4倍/年、存储器带宽每6年增长2倍，滞后于AI算力需求，为主要性能瓶颈

模型规模指数上升，存储容量成本瓶颈



存不下

硬件算力增速滞后于AI算力需求

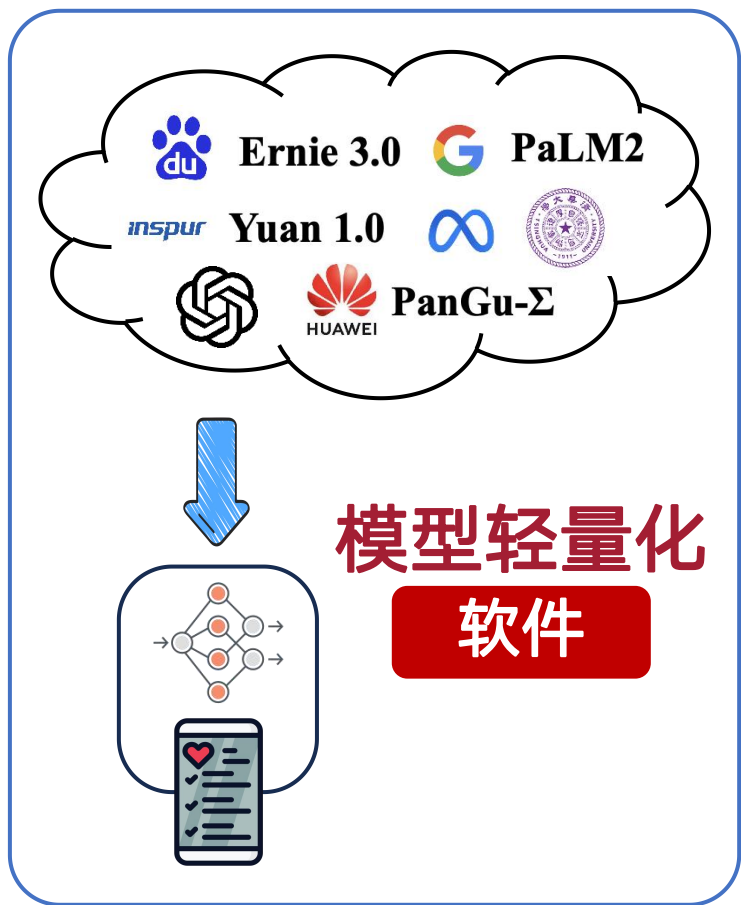


算不快

用不满

迈向通用智能的关键：协同智能

- 充分利用端、云侧的**泛在资源**，建立大模型**协同计算体系**，弥补端侧大模型的性能衰减，实现高效的**端云智能体构建与协同**



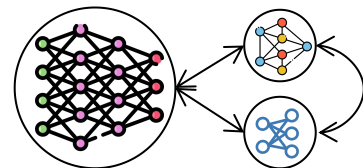
端侧模型部署



端云协同计算

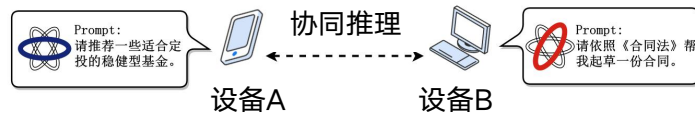
云侧

协同微调提升**部署模型性能**



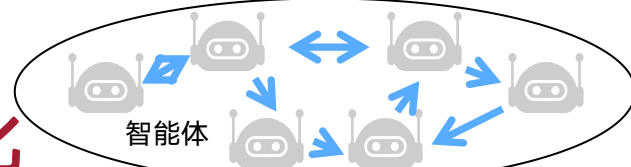
云/端

协同推理提升**计算效率与精度**



端侧

协同决策提升**智能体泛化能力**

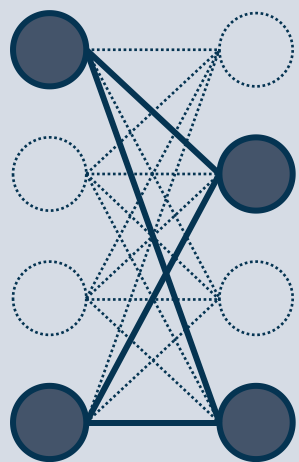


范式轻量化

硬件

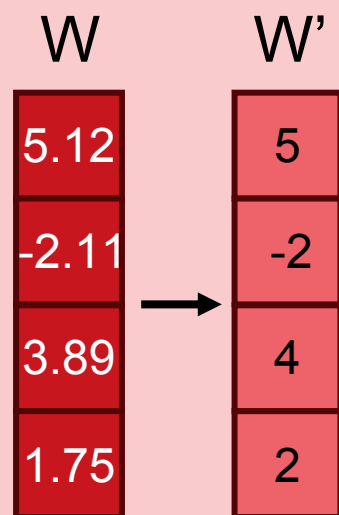
缓解部署压力的通用技术

剪枝



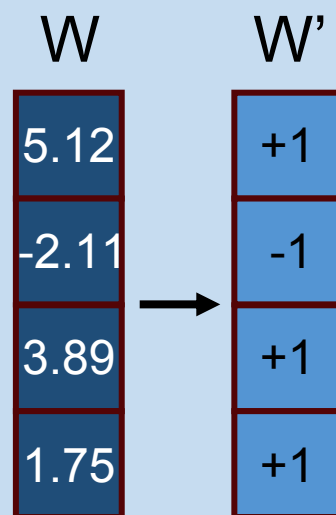
删去不必要连接^[10]

量化



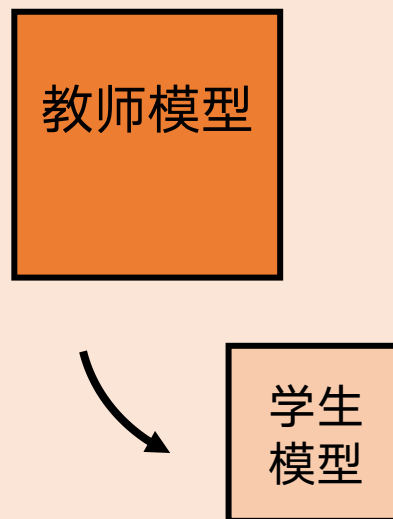
降低数值精度^[11]

二值化



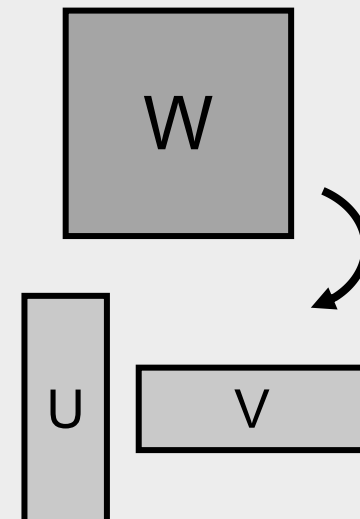
只使用1-bit值
(-1 or +1) ^[12]

知识蒸馏



将知识迁移到小模型
^[13]

低秩分解



将大矩阵分解为小因子矩阵^[14]

[10] Hoefler, Torsten, et al. "Sparsity in deep learning: Pruning and growth for efficient inference and training in neural networks." JMLR 22.241 (2021).

[11] Nagel, Markus, et al. "A white paper on neural network quantization." arXiv preprint arXiv:2106.08295 (2021).

[12] Zhang, Baochang, et al. "Binary neural networks: Algorithms, architectures, and applications." CRC Press (2023).

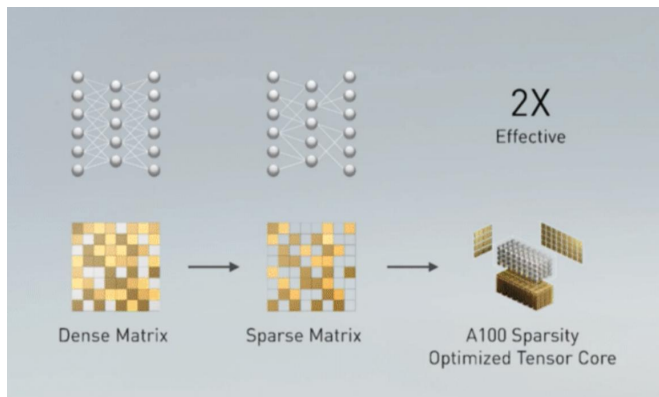
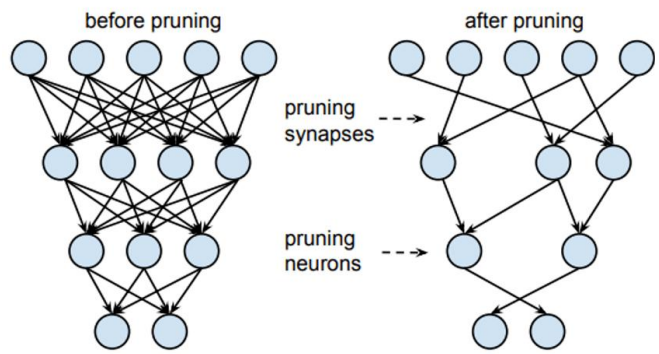
[13] Gou, Jianping, et al. "Knowledge distillation: A survey." IJCV 129.6 (2021).

[14] Liu, Xingyi, and Keshab K. Parhi. "Tensor decomposition for model reduction in neural networks: A review [Feature]." IEEE CAS Magazine 23.2 (2023).

基础：模型“轻量化”——模型压缩

模型稀疏化

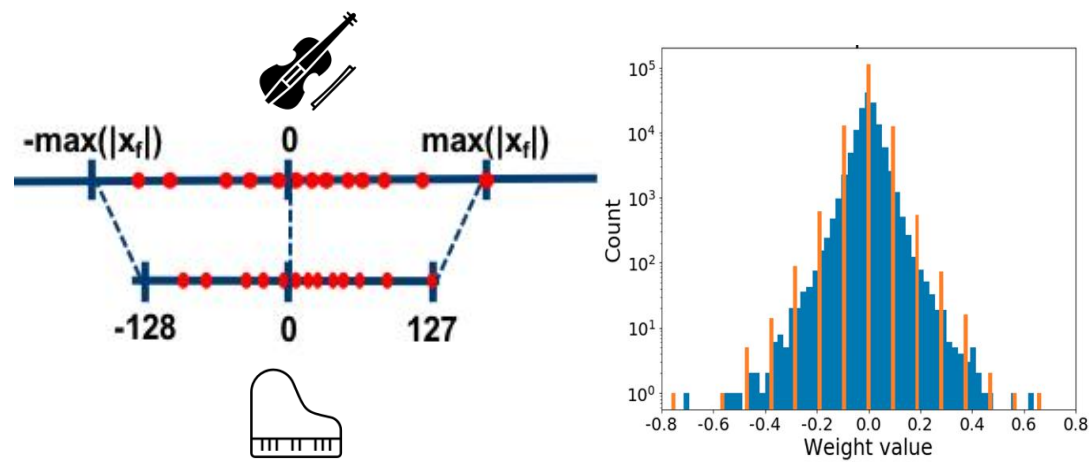
DNN模型包含大量的冗余，移除不重要的权值



Ampere GPU [Source: NVIDIA]

数值量化

用更低精度来表示操作数，简化算子执行计算



Peak FP64 ¹	9.7 TFLOPS
Peak FP64 Tensor Core ¹	19.5 TFLOPS
Peak FP32 ¹	19.5 TFLOPS
Peak FP16 ¹	78 TFLOPS
Peak BF16 ¹	39 TFLOPS
Peak TF32 Tensor Core ¹	156 TFLOPS 312 TFLOPS ²
Peak FP16 Tensor Core ¹	312 TFLOPS 624 TFLOPS ²
Peak BF16 Tensor Core ¹	312 TFLOPS 624 TFLOPS ²
Peak INT8 Tensor Core ¹	624 TOPS 1,248 TOPS ²
Peak INT4 Tensor Core ¹	1,248 TOPS 2,496 TOPS ²

1 - Peak rates are based on GPU Boost Clock.
2 - Effective TFLOPS / TOPS using the new Sparsity feature

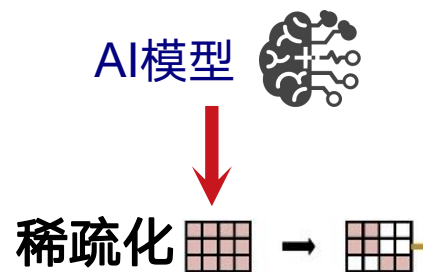
A100

核心挑战

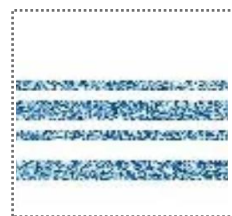
AI模型



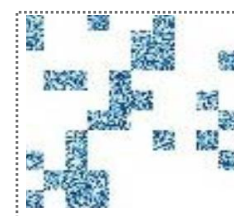
核心挑战



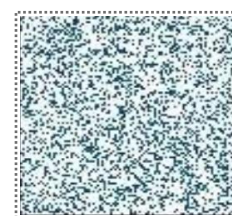
列级稀疏



行级稀疏

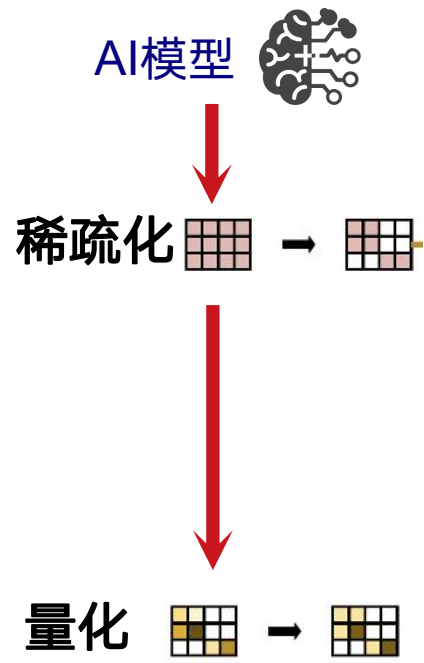


块级稀疏

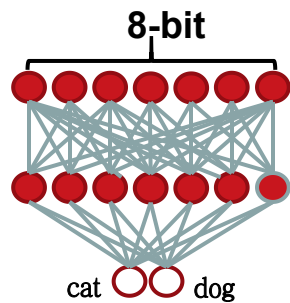


元素级稀疏

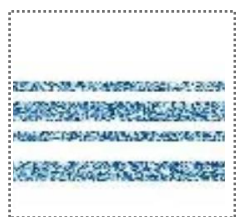
核心挑战



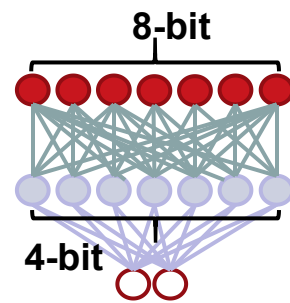
列级稀疏



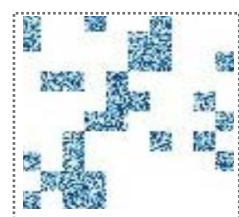
全网一致量化



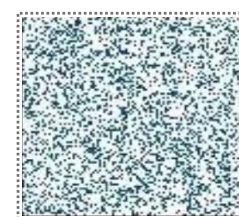
行级稀疏



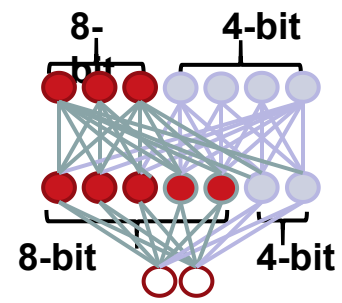
层间混合量化



块级稀疏

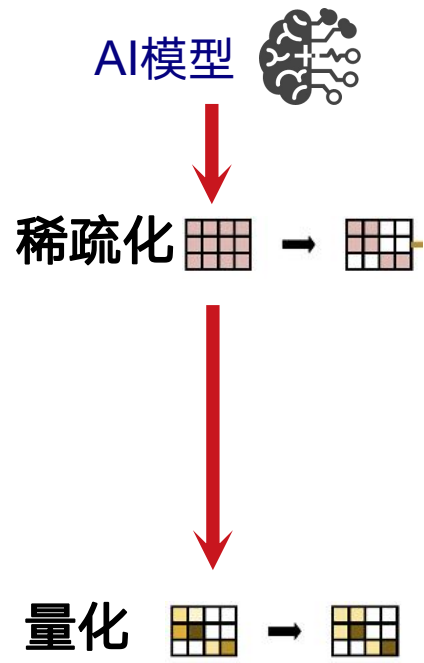


元素级稀疏

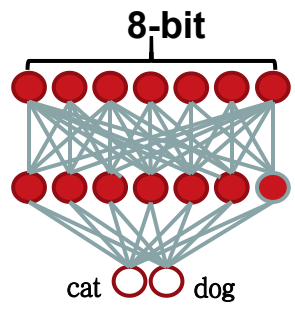


层内混合量化

核心挑战



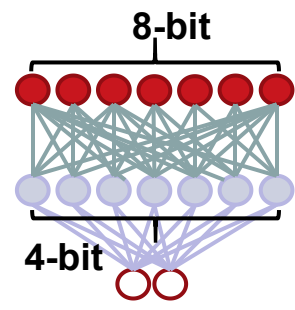
列级稀疏



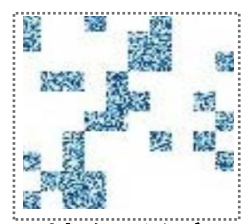
全网一致量化



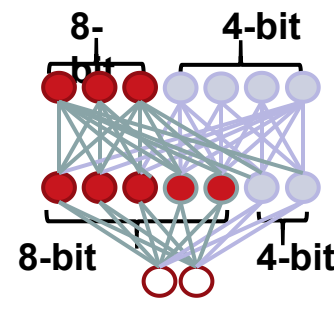
行级稀疏



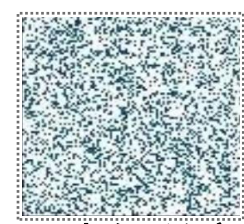
层间混合量化



块级稀疏



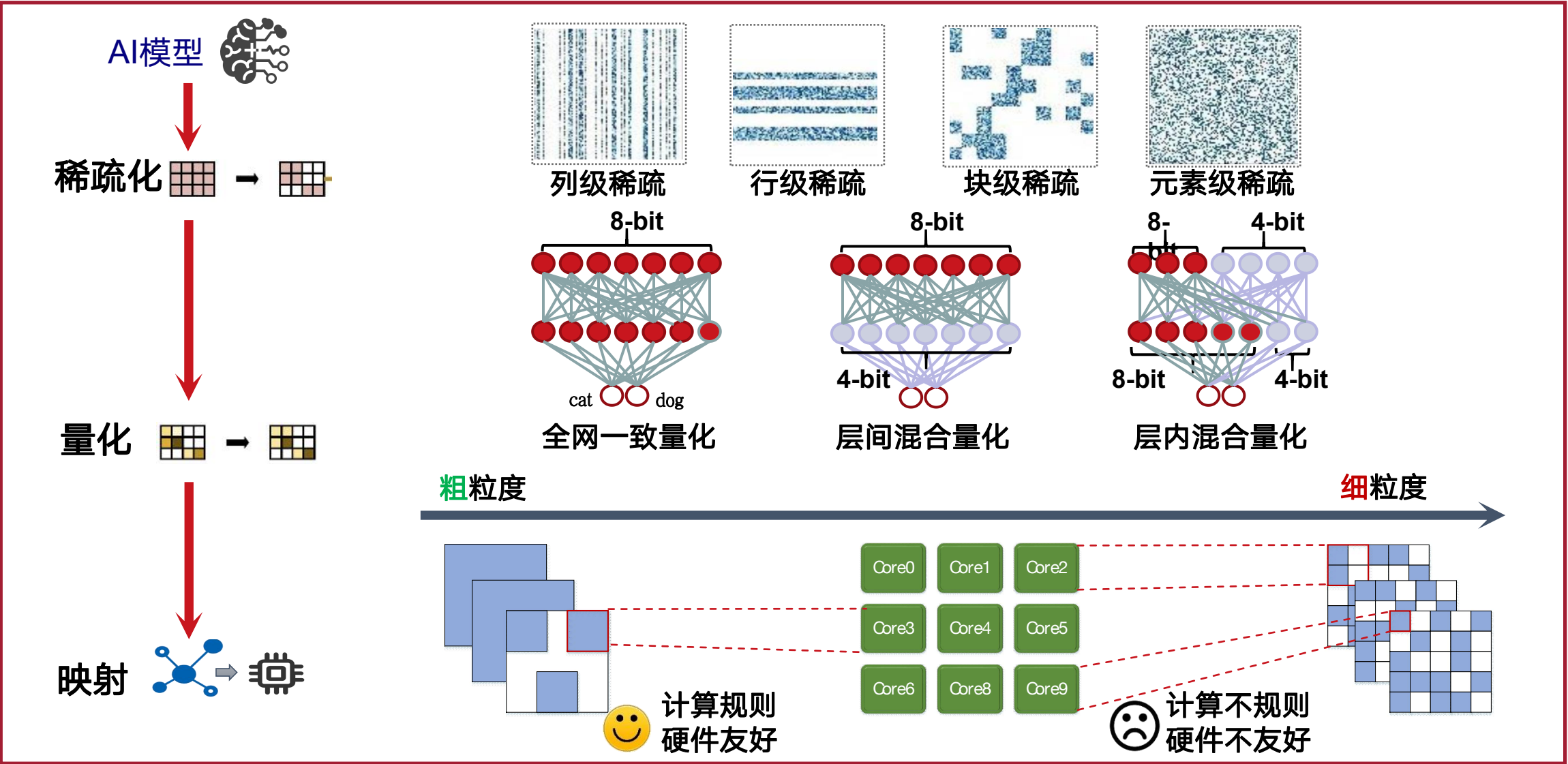
层内混合量化



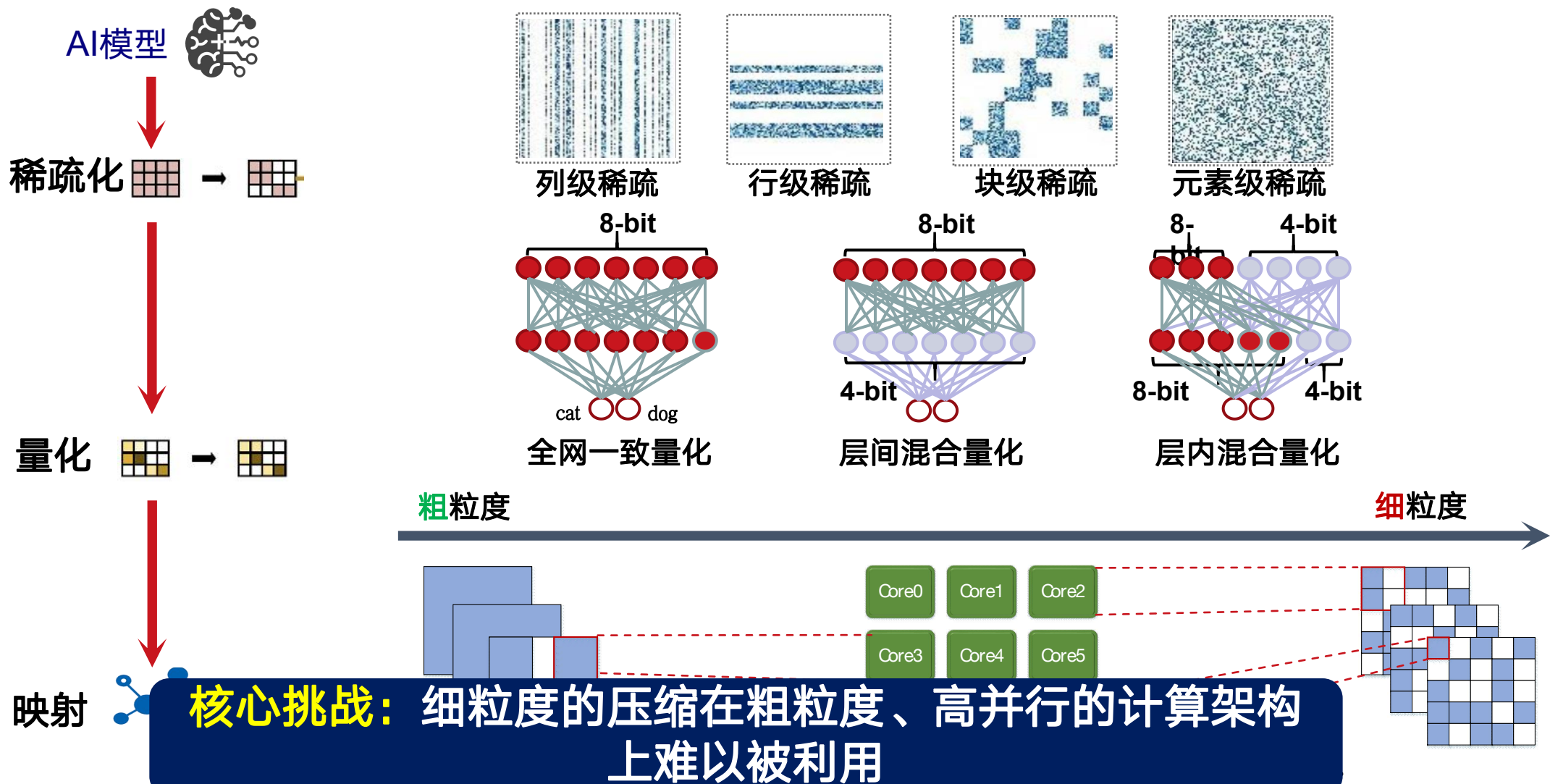
元素级稀疏



核心挑战



核心挑战

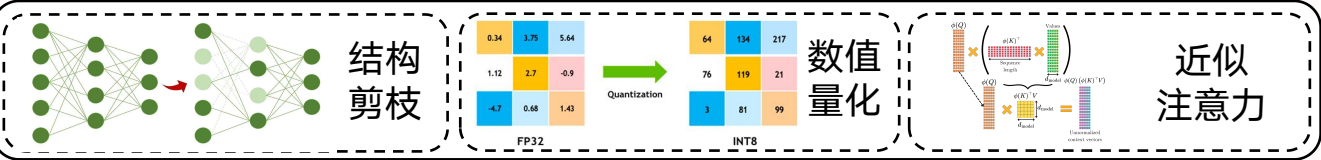


研究视野

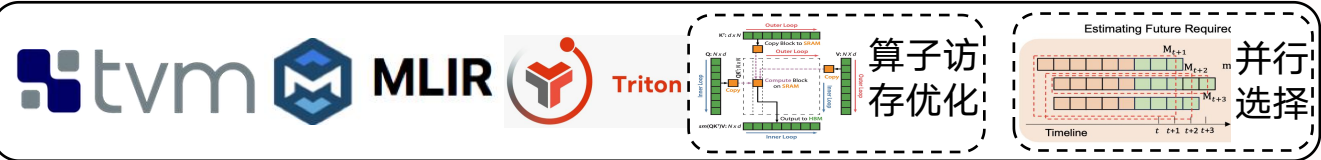
智能模型



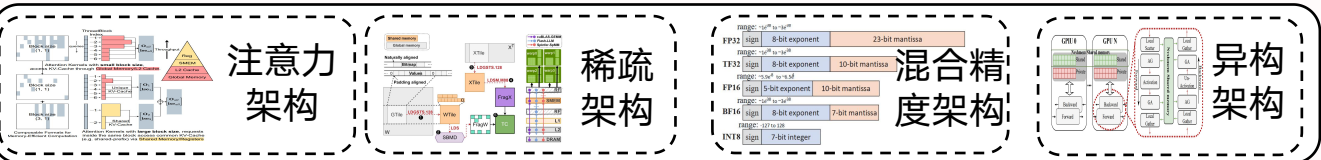
算法空间



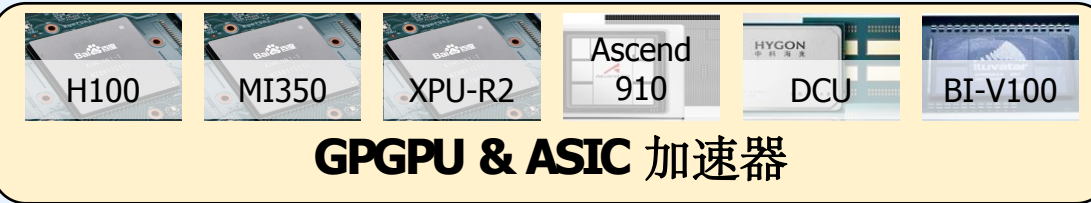
系统/编译



架构空间



多样化硬件



辅助工具



面向生成式 AI 的异构计算轻量化加速技术

生成式模型异构 (Model Heterogeneity)



极大规模 LLM/MoE: 显存墙严峻、动态稀疏性、长序列 KV Cache 瓶颈

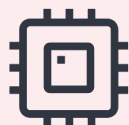


实时生成式渲染 (NeRF/3DGS): 高帧率要求、计算负载不均衡、对专用加速器适配差

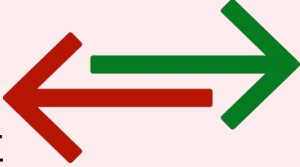
算力硬件异构 (Compute Heterogeneity)



云端高性能池 (GPU/TPU): 追求吞吐量, 互联带宽是瓶颈

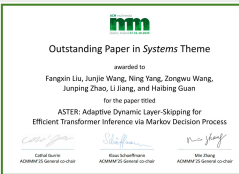


边缘/端侧设备 (NPU/CPU): 功耗与面积受限, 缺乏软件栈支持



(算法层) 极致量化动态推理

- [HPCA'24] SPARK: 高位冗余编码压缩加速
- [DAC'25 | 中兴落地] BLOOM: 异常值比特切分
- [AAAI'26] SpecQuant: 极低比特量化压缩
- [ICCAD'25] QUARK: 非线性算子低精度计算



- [ASPLOS'25] EARTH: 熵感知的 MoE 投机预取
- [ISCA'25] FATE: 类脑的自适应数据格式原型
- [ACM MM'25 | 杰出论文] ASTER: 动态跳层
- [EMNLP'25 | 蚂蚁落地] FlexQuant: 精度切换框架

(系统层) 系统弹性调度优化

- [DAC'25 | 华为落地] MILLION: KV异常值量化
- [TACO'25] SPADE: 比特稀疏性优化并行解码
- [ICCAD'25] PLAIN: PIM 与混合精度协同
- [TCAD'24] SpMMPlu-Pro: 稀疏编译框架



- [技术报告 | Huawei] HyperOffload: 卸载计算图算子
- [HPCA'25] CROSS: 负载感知稀疏与密集融合 kernel
- [PPoPP'26] BEEMS: 内存“削峰填谷”生命周期管理
- [TACO'25] MIX-PC: 混合数值精度编译优化 LLM 部署

(硬件层) 异构算力池化加速

- [DAC'25] ALLMod: 查表迭代混合数据流
- [TCAD'25] DevTrace+: 驱动事务跟踪插桩工具
- [ASPLOS'26] Nebula: CPU/GPU 协同的 3DGS
- [LSI'26] 特别设计 | 以查强算端侧加速芯片



- [ISCA'24] UM-PIM: 统一 DRAM-PIM 共享内存架构
- [MICRO'24] COMPASS: 自适应投机 SRAM-CIM 架构
- [ASPLOS'26] PUSHtap: PIM 统一存储的 HTAP 架构
- [HPCA'26] ORANGE: 平衡任务分配 NPU 架构优化

形态之简

位宽与深度的动态“剪裁”

“解决“存不下”的问题。

对模型精度（位宽）和计算深度（层数）做减法，
让数据以最精简的形态存在。”

ASTER: Transformer 自适应动态层跳层框架

[ACM MM 2025]

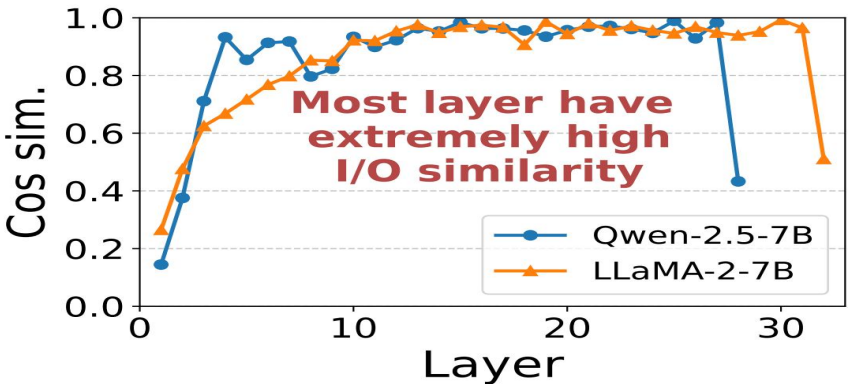
洞察：LLM存在层间信息冗余

- 对于LLM，以LLaMA-2-7B 和 Qwen2.5-7B-Instruct为例，我们发现大部分层的输入和输出存在极高的相似性

$$Similarity(\vec{a}, \vec{b}) = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \|\vec{b}\|} = \frac{\sum_{i=1}^n a_i b_i}{\sqrt{\sum_{i=1}^n a_i^2} \sqrt{\sum_{i=1}^n b_i^2}}$$

$$Mean(Similarity)_j = \frac{\sum_{i=1}^N \sum_{t=1}^{|T_i|} Similarity(a_{it}, b_{it})}{\sum_{i=1}^N |T_i|}$$

相似性度量来判断线性层的重要性



大量层的输入和输出相似性高，这些层的信息含量低

现有方案

方法	代表工作	核心问题
跳层	Early Exit、AdaSkip	难以平衡跳过率与预测质量，依赖浅层特征 / 静态规则，跨模态适应性差
知识蒸馏	TinyBERT、MiniLM	主要用于模型压缩，未充分探索其补偿层跳过信息损失的潜力



缺乏自适应性和信息丢失风险

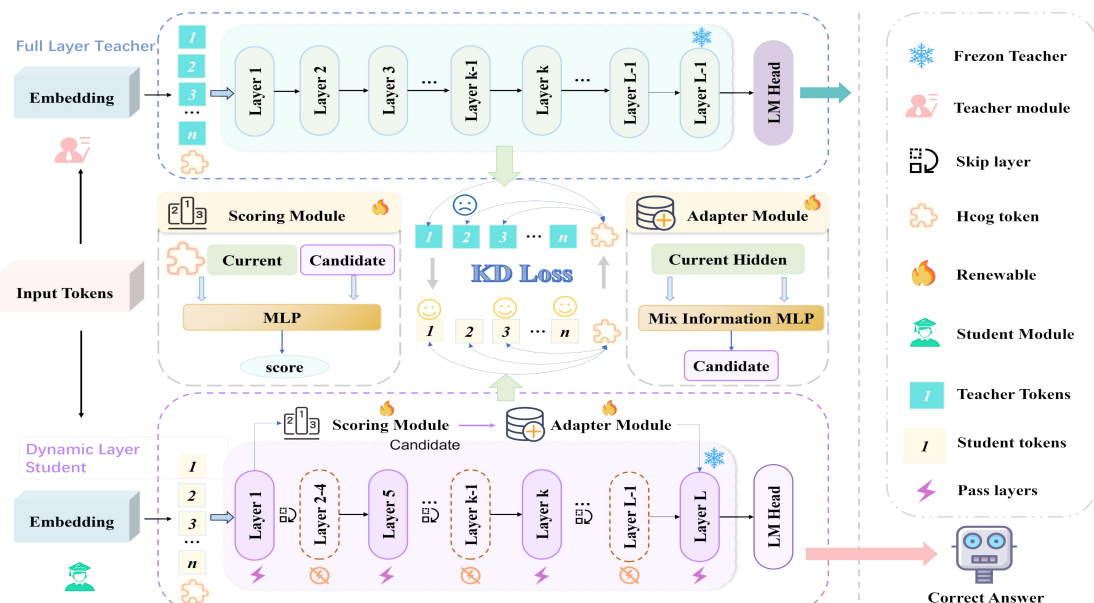
ASTER: Transformer 自适应动态层跳过框架

启发：向人脑学习

基于当前所处环境和状态作出当前最佳决策

静态预定义跳层

动态自适应马尔可夫决策过程



推理时融入蒸馏引导信号，缓解跳层的精度损失

设计思路：借鉴 ViT 的 [CLS]，引入令牌追踪计算状态：

- 编码器（BERT/ViT）：复用 [CLS] token
- 解码器（LLaMA）：用输入最后 1 个 token 作 <cognitive> token

执行过程

- ① 输入预处理与认知令牌注入：初始化模型参数 / 状态，注入认知令牌增强特征表达
- ② 动态层决策与执行：生成有效候选层
- ③ 层跳过评分与选择：基于特征相似性等指标评分，确定最终跳层方
- ④ 动态适配器适配（若跳层）：实现维度映射 + 分布对齐，解决跳层后前后层特征衔接断裂问题
- ⑤ 知识蒸馏辅助训练：用教师模型（完整层）补偿跳层带来的信息损失，平衡推理速度与模型精度

- 动态跳层建模为序列决策问题
- token级表示适配输入动态性

ASTER: Transformer 自适应动态层跳过框架

[ACM MM 2025]

实验结果

Method	ViT-Base (ImageNet)			LLAMA3.1-8B (Boolq)		
	1.5×	2.0×	3.0×	1.14×	1.33×	2.0×
Baseline (full layers)	81.32%			72.48%		
Early-exit	19.36%	8.93%	2.64%	65.35%	62.17%	57.98%
random-skip	20.37%	0.53%	0.15%	44.72%	0.28%	0.01%
Skiptocode	33.45%	2.64%	0.50%	45.02%	9.05%	1.41%
Shortgpt	14.39%	13.41%	3.74%	71.90%	61.33%	40.90%
Ours	44.46%	32.45%	25.80%	66.33%	64.55%	64.50%

ViT模型

- ViT-Base模型在ImageNet数据集上的结果：
 - 1.5倍加速**：ASTER达到44.46%准确率，比第二名Skiptocode(33.45%)高出约11个百分点
 - 2.0倍加速**：ASTER保持32.45%准确率，而第二名Shortgpt仅有13.41%
 - 3.0倍加速**：ASTER仍能保持25.80%准确率，其他方法都低于4%

LLaMa3模型

- LLAMA3.1-8B模型在BoolQ数据集上的结果：
 - 1.14倍加速**：Shortgpt表现最好(71.90%)，ASTER排名第三(66.33%)
 - 1.33倍加速**：ASTER(64.55%)超过Early-exit(62.17%)和Shortgpt(61.33%)
 - 2.0倍加速**：ASTER保持相对稳定的性能(64.50%)，显著优于第二名Early-exit(57.98%)

方案小结

提出解决方案： ASTER —— 一种面向大语言模型的动态跳层推理框架。

核心目标： 通过减少推理过程中的计算冗余，加速大语言模型的推理过程。

双组件协同优化策略：

- a. 策略网络：基于模型的实时状态，自适应地学习并决定最优的计算路径。
- b. 状态适配器：结合知识蒸馏进行训练，确保跨层特征的平滑过渡，并补偿跳层过程中产生的信息损失。

最终优势： 通过动态分配计算资源，实现输入自适应的推理机制，在推理速度与模型精度之间取得更加优越的权衡。

方案小结

提出解决
核心目标
双组件协
a. 策
b. 状
偿
最终优势
模型精度

ACM multimedia



Dublin, Ireland 27-31.10.2025

Outstanding Paper in *Systems* Theme

awarded to

Fangxin Liu, Junjie Wang, Ning Yang, Zongwu Wang,
Junping Zhao, Li Jiang, and Haibing Guan

for the paper titled

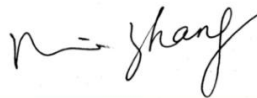
ASTER: Adaptive Dynamic Layer-Skipping for
Efficient Transformer Inference via Markov Decision Process



Cathal Gurrin
ACMMM'25 General co-chair



Klaus Schoeffmann
ACMMM'25 General co-chair



Min Zhang
ACMMM'25 General co-chair

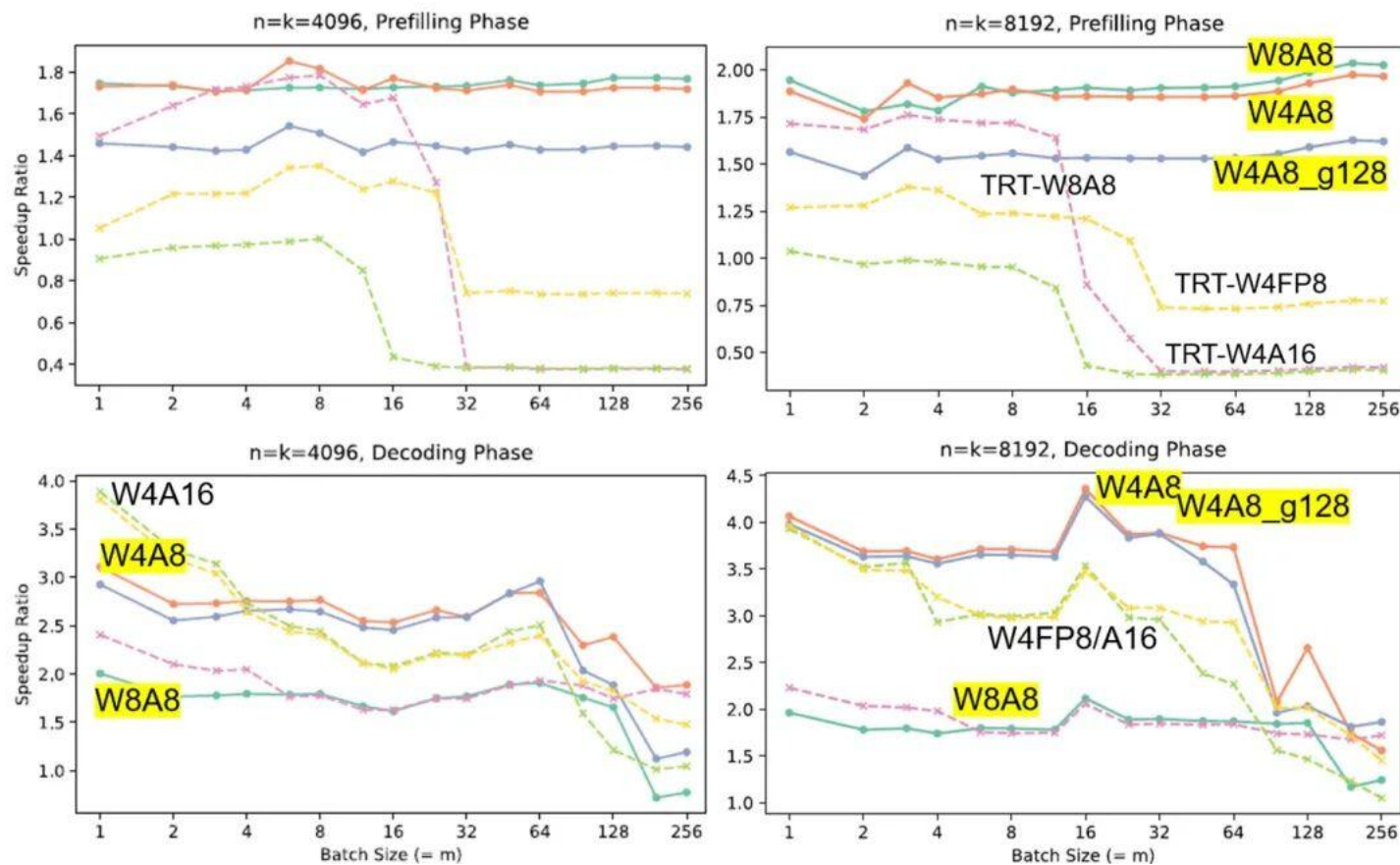
框架。
过程。
的计算路径。
过渡，并补
推理速度与

输入难易度感知的自适应精度推理策略

“依据输入样本复杂度动态调整计算位宽，从数值精度层面实现推理成本的精细化压缩。”

核心观察

位宽分配受负载影响



不同量化方法的速度对比：PD阶段和batch

观察

P阶段：

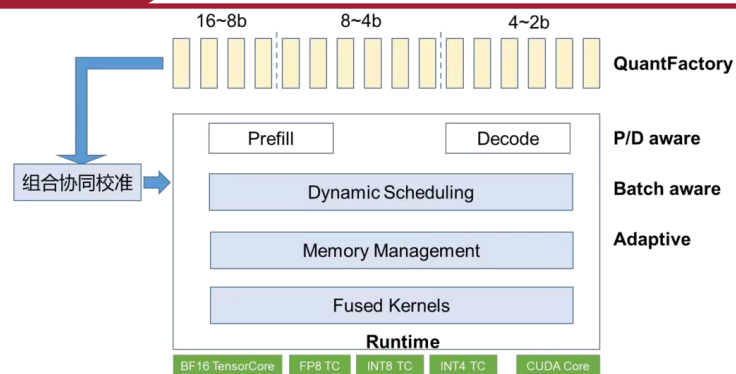
➢ W8A8和W4A8明显最快（中大batch）

D阶段：

➢ 小batch下、中小模型：W4A16优

➢ 大batch下、更大模型，W4A8更好

PD感知



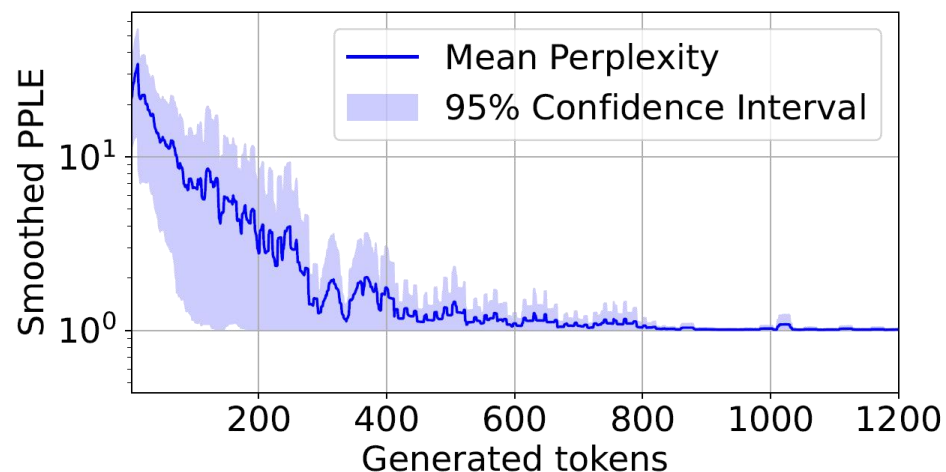
P阶段 W8A8，D阶段 W4A16、W4A8

BS小

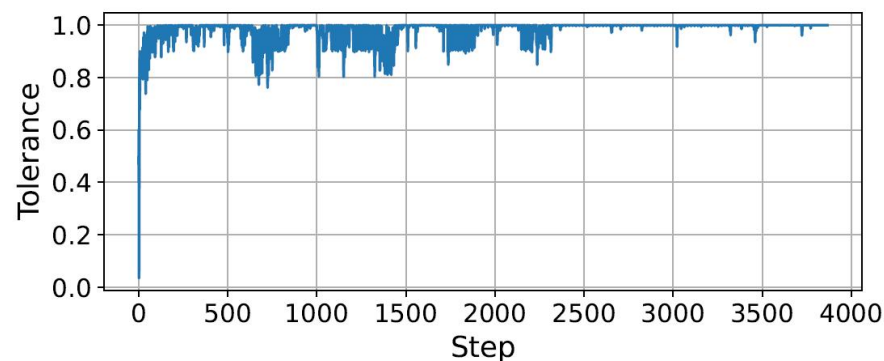
BS大

核心观察

核心观察：随着生成长度的增加，模型输出置信度呈现一致的上升趋势。



核心观察：模型对误差的容忍度随输出长度增加而提升。



(a) Fault tolerance of Llama2-7B.

在可控精度损失范围内，模型输出token可保持稳定，同时获得速度与内存效率的提升。

方案总览

整体方案架构

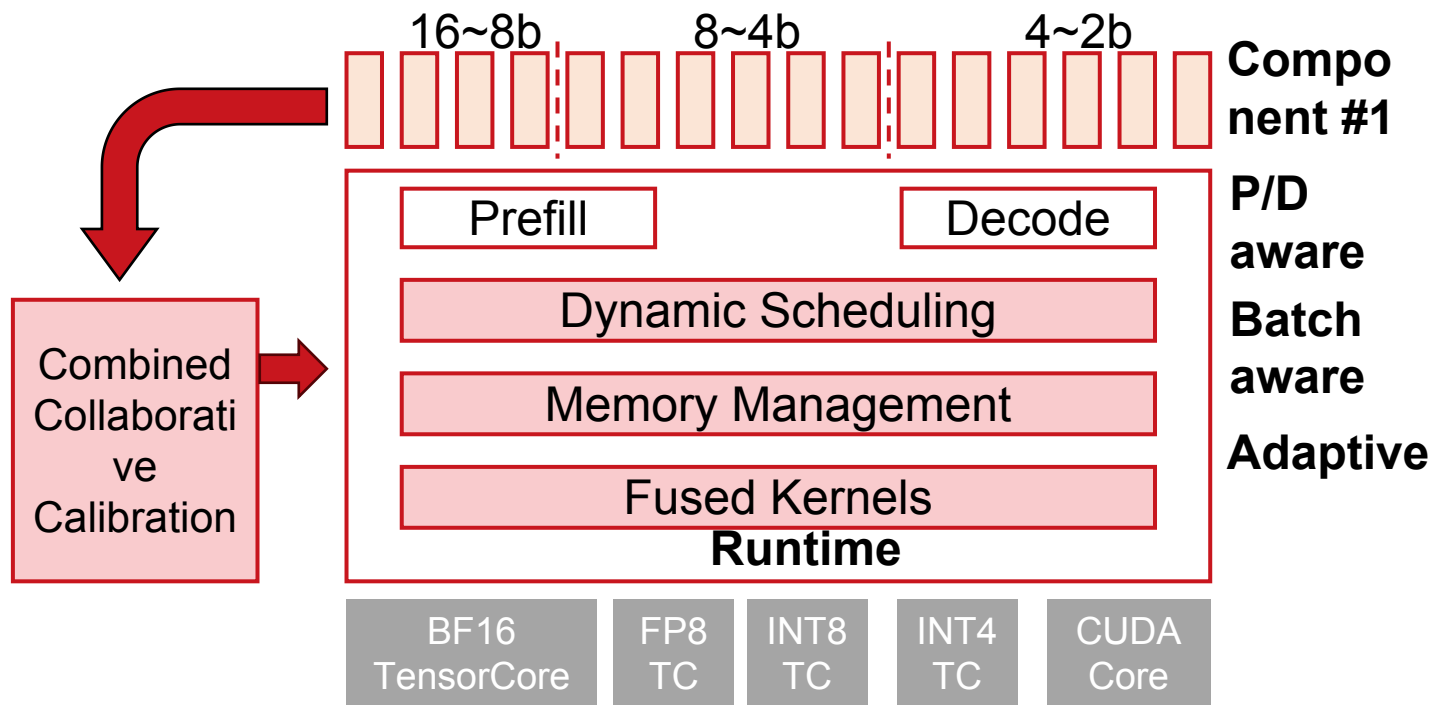
- 2 个核心组件: 量化模型生成 (16/8/4位) + 动态调度 (预填充/解码/批次/层级感知)
- 底层框架: vLLM (兼容SGLang)

组件 1: 多版本量化模型

- 功能: 为后续调度提供多样化的量化模型
- 支持: 16-bit, 8-bit, 4-bit等位宽。

组件2: 动态调度

- 核心逻辑: 基于预填充/解码阶段、批次及层级特性动态调整量化模型
- 优势: 在速度、内存与生成质量间实现平衡



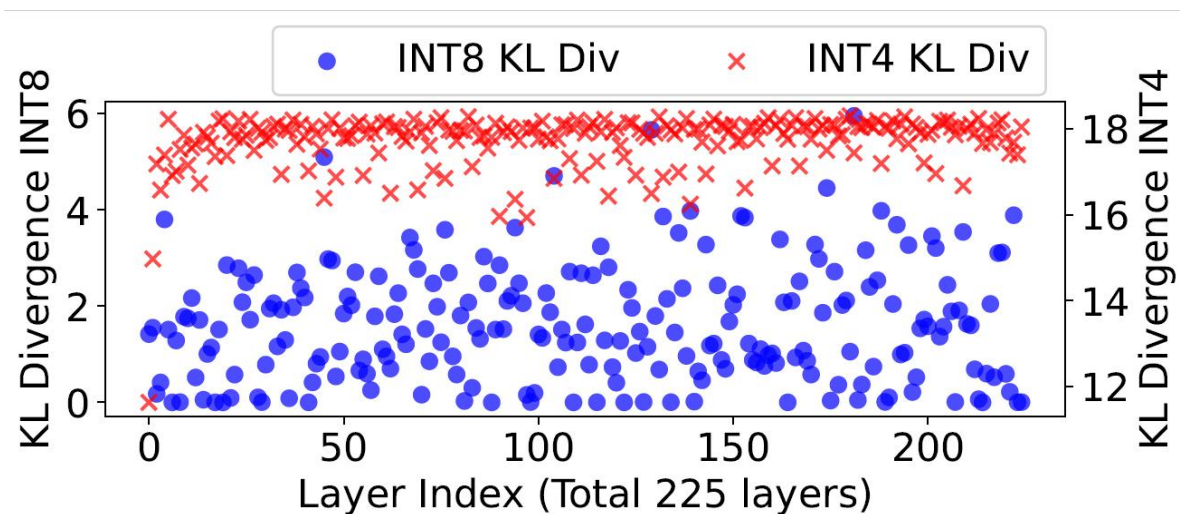
方案优化

核心挑战

- 不同量化位宽在各模型层间存在显著的精度差异（例如，线性层2从16位量化降至4位时出现明显的KL散度下降）。

优化目标

- 开发可靠的调度算法以缓解精度损失 → 提出**FlexQuant生成算法**。



方案优化

FlexQuant 算法核心步骤

- **敏感度排序:** 利用小规模数据集计算线性层在不同量化位宽下的KL散度差异 $\rightarrow$ 排序各层对量化的敏感度。
- **动态替换机制**
 - 生成过程中持续监测模型的生成概率困惑度
 - 当困惑度达到阈值时: 基于先前的敏感度排序, 将指定线性层替换为更低比特量化版本。

Algorithm 1 FlexQuant Dynamic Precision Inference

Input: Prompt P of n tokens

Output: Sequence S

$\text{logit}_{n-1} \leftarrow \text{Prefill}(P)$ with WxAy model

$T_n \leftarrow \text{argmax}(\text{logit}_{n-1})$

$S \leftarrow \text{concat}(P, T_n)$

$\text{thre} \leftarrow \text{ave}(\exp(\text{entropy}(\text{logits})))$

for $i = 0$ **to** M **do**

$\text{logit}_{n+i} \leftarrow \text{Decoding}(S)$

$T_{n+i+1} \leftarrow \text{argmax}(\text{logit}_{n+i})$

$S \leftarrow \text{concat}(S, T_{n+i+1})$

$PPLE \leftarrow \exp(\text{entropy}(\text{logit}_{n+i}))$

if $\text{slide_ave}(PPLE) < \text{thre}$ **then**

Replace Wx with lower bit-width of specific- Linear layers

end if

end for

实验结果

长文本生成场景

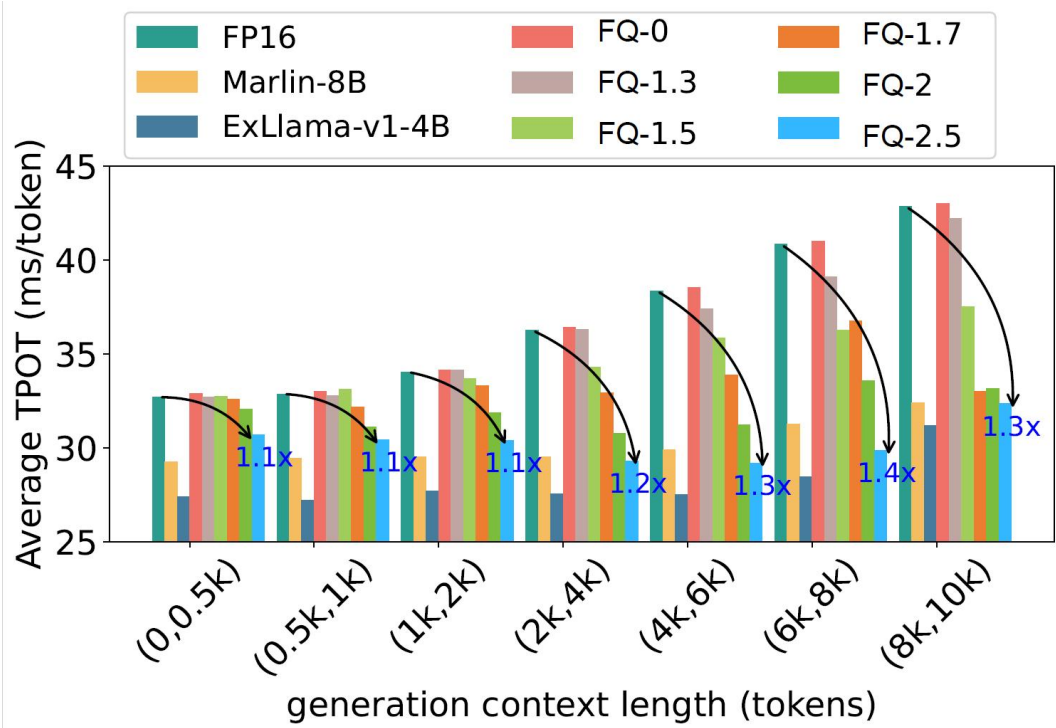


Table 2: Accuracy evaluation of FlexQuant on LongWriter Evaluation Set.

Bits	FP16	INT-8	INT-4	FQ-1.3	FQ-1.5	FQ-1.7	FQ-2.0	FQ-2.5
Rouge-L	100	46	26	89.43	75.44	66.58	60.86	56.14
DeepSeek-V3 Score	89.02	87.63	81.66	88.54	88.19	89.23	88.33	87.6

精度性能表现

- FlexQuant: 优于INT8量化精度
- FQ1.3: 基本保持原始模型精度水平

速度性能表现

- 在6k token生成长度下：相比原始模型实现1.4倍端到端加速。

实验结果

传统生成任务场景

Model	Method	CNN/DM			Dialogsum			IWSLT		
		Bits	Rouge-L	BERTScore	Bits	Rouge-L	BERTScore	Bits	Rouge-L	BLEUScore
Vicuna-7B	Baseline-l	2	4.29	76.74	2	1.57	73.5	2	2.24	0
	Baseline-h	3	22.27	87.01	3	23.57	87.56	3	52.98	25.65
	DNS*	2.39	22.27	86.92299	2	-	-	2.68	-	22.39245
	PMPD-Static*	2.39	22.35908	87.09701	2	24.15925	87.56	2.68	-	25.2396
	PMPD-Learned*	2.39	22.09184	86.83598	2.74	23.66428	87.56	2.37	-	24.2649
	FlexQuant	2.56	20.04	85.24	2.74	23.2	87.23	2.68	52.41	20.91
MobileLlaMA	Baseline-l	3	9.3	78.93	3	4.42	78.48	3	6.35	0.39
	Baseline-h	4	14.64	83.99	4	10.32	81.48	4	19.89	4.51
	DNS*	3.37	14.81568	83.99	3.21	9.03	80.99112	3.65	-	4.26195
	PMPD-Static*	3.37	14.97672	84.15798	3	10.44384	81.56148	3.65	-	4.47392
	PMPD-Learned*	3.37	14.1276	83.65404	3.21	10.50576	81.56148	3.48	-	4.18979
	FlexQuant	3.37	16.36	84.31	3.37	10.57	81.65	3.5	24.65	6.83
Phi-1.5/ Zephyr-3B**	Baseline-l	3	7.06	77.72	3	7.41	80.79	3	11.82	1.96
	Baseline-h	4	12.27	83.31	4	9.4	81.87	4	55.01	24.31
	DNS*	3.71	9.38655	81.14394	3.3	8.4036	81.46065	3.34	-	22.55968
	PMPD-Static*	3.71	12.27	83.31	3.3	9.4564	81.95187	3	-	23.75087
	PMPD-Learned*	3.09	11.74239	82.72683	3.52	9.3436	81.78813	3.34	-	23.8238
	FlexQuant	3.11	12.73	83.18	3.52	10.86	82.86	3.44	54.58	22.88

* Indicates that the data is scaled by the results in PMPD to the baseline-h of this paper.

** The Phi-1.5 model is used to evaluate the CNN/DM and Dialogsum tasks, and the Zephyr-3B model is used to evaluate the IWSLT task.

整体性能表现

- FlexQuant在传统文本生成任务中保持了可观的精度和等效位宽。

具体模型结果

- 最优 PMPD结果: Vicuna-7b.
- FlexQuant综合最优: MobileLlaMA, Phi-1.5以及 Zephyr-3B.

方案小结

方案: FlexQuant — 面向大语言模型的动态token级量化框架

核心目标: 弥合大模型内存需求与现代硬件能力之间的鸿沟

双重优化机制:

- a. Token级: 采用困惑度熵指导细粒度精度调度
- b. 层间: 基于KL散度的优化确保全局一致性

最终优势: 现自适应高效量化，精度损失最小化 → 适用于大语言模型实时推理

方案小结

方案
核心
双重
最终

FlexQuant: 大模型组合量化，助力推理SLO

原创 AI infra AI Infra 2025年08月29日 20:17 湖南

- 基于本方案，与上海交大合作完成的同名论文已被AI顶会接收。arxiv链接：

FlexQuant: A Flexible and Efficient Dynamic Precision Switching Framework for LLM Quantization

<https://arxiv.org/abs/2506.12024>

文章作者：李金金，赵守仁，蔡腾纬，赵军平。

介绍蚂蚁集团Asystem团队、在线推理团队针对大模型推理量化加速的探索实践，提出的FlexQuant方案通过组合业界多种量化方法，且在运行时动态协同、取长补短以满足业务对精度、延迟和吞吐等全方面诉求，效果反馈良好，且实测发现1+1>2意外之喜。此外，该方案可完美匹配PD分离场景。欢迎交流探讨。

理

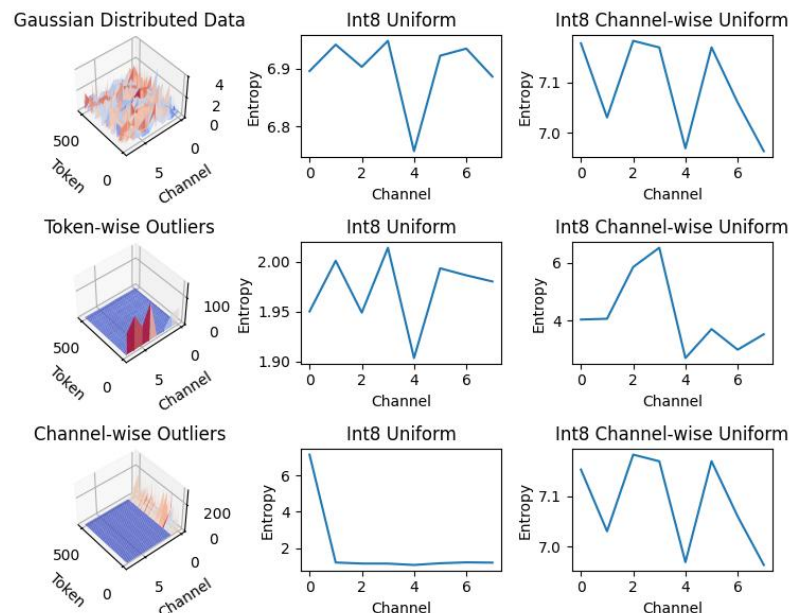
蚂蚁上线部署：大模型推理生成速度加速~40%；吞吐能力对比业界同类方案提升超过~60%

MILLION: 基于PQ的KV压缩与推理加速

[DAC'25]

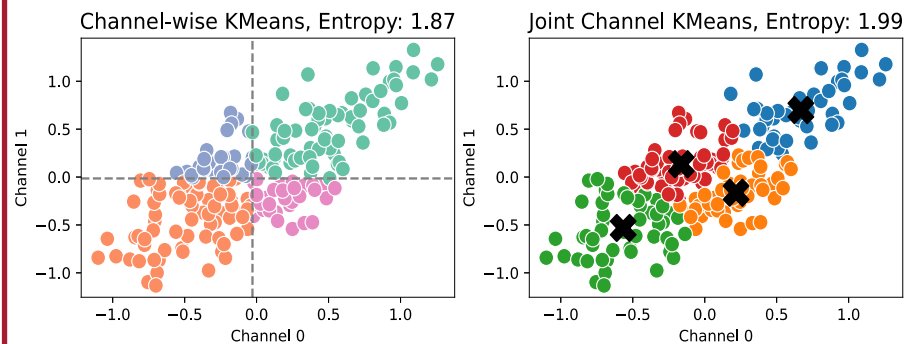
华为2012实验室合作项目

关键洞察

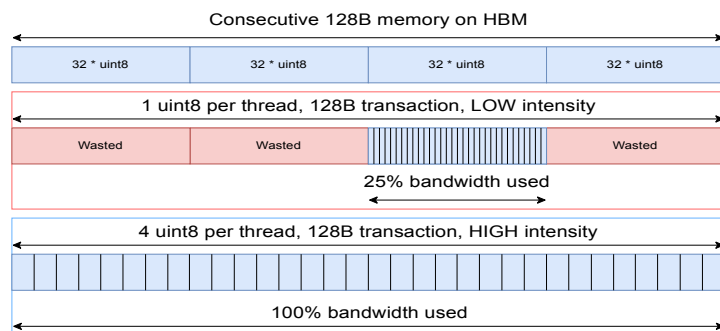


- KV-cache中，不同通道离群值分布有显著差异
- 沿特定维量化无法应对复杂分布
- 混合数据编码尚未得到通用支持
- 是否存在一个纯软件方案？

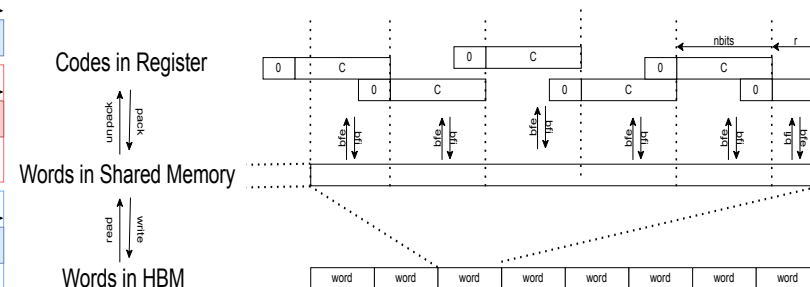
核心思想



合并PQ码本，有效利用内存带宽（25%→100%）



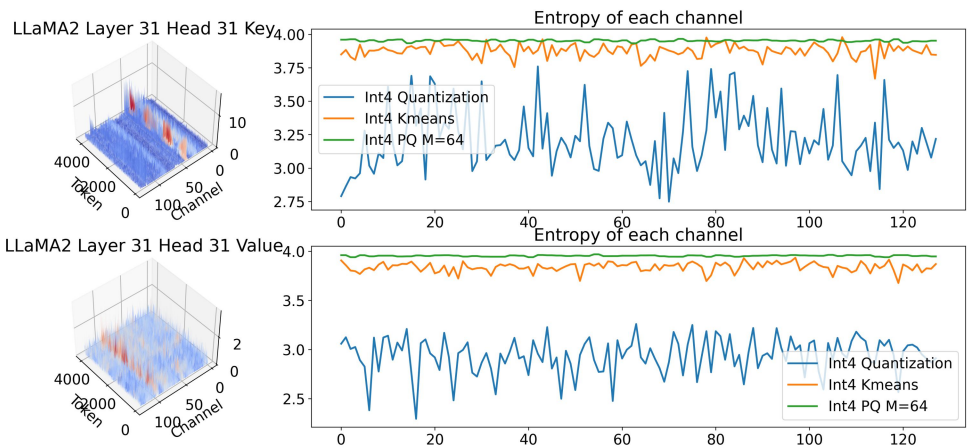
低位宽码本@shared memory
编写PTX指令完成bit操作



可根据上下文长度(Token length)动态调整并行策略

MILLION: 基于PQ压缩的以查代算PIM友好框架

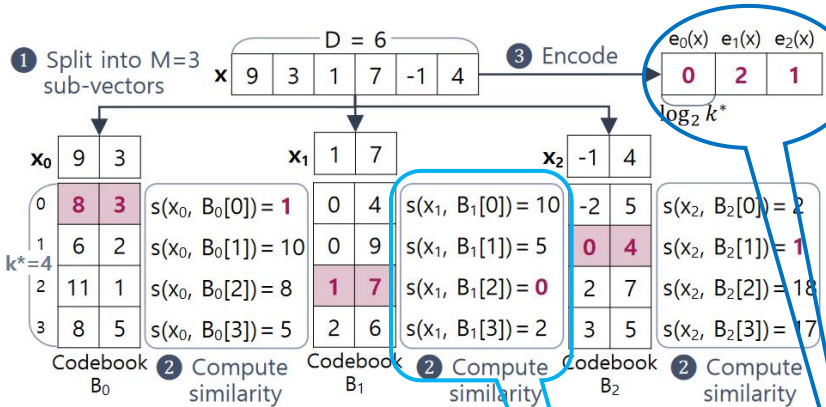
跨通道量化能够显著提升信道容量利用率



高精度3~4比特超低位宽KV量化

PERPLEXITY EVALUATION RESULTS ON VARIOUS MODELS ACROSS DIFFERENT DATASETS.

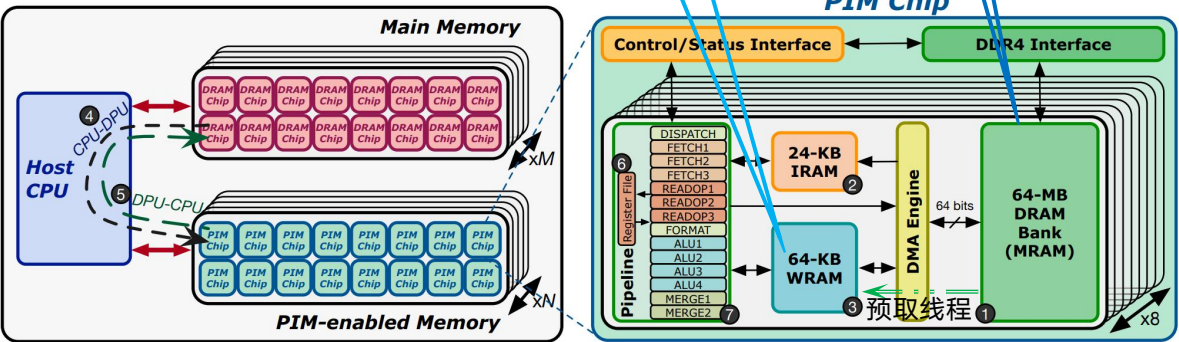
Method	GPT2-xl		LLaMA-2-7B		MPT-7B	
	Wiktext-2	PTB	Wiktext-2	PTB	Wiktext-2	PTB
baseline	17.41	21	5.12	28.31	7.68	9.98
KVQuant-3b	18.59	23	11.21	12323.75	27681432	3E+07
KVQuant-3b-1%	18.24	22.35	5.22	24.34	7.826	10.14
MILLION-3b	17.6	21.14	5.2	29.55	7.79	10.08
KVQuant-4b	18.156	22.32	6.99	102.21	24043348	3E+07
KVQuant-4b-1%	18.108	22.15	5.14	25.86	7.71	10.02
MILLION-4b	17.57	21.12	5.21	29.56	7.75	10.03



PQ算法示意图

准静态码本

静态数据



以查代算近存计算框架

优点

- 超低位宽量化优化访存瓶颈
- PQ以查代算缓解近存算力瓶颈（无FP ALU需求）
- 单DPU实现0.8TOPS等效算力

时间之准

从“被动验证”到“主动预测”

“解决“传输慢/等得久”的问题。

打破传统的被动执行，让数据提前出现在流水线中。”

CSD:通过语义校准打破投机解码“僵化验证”

[ACL 2026 Main

大模型解码瓶颈

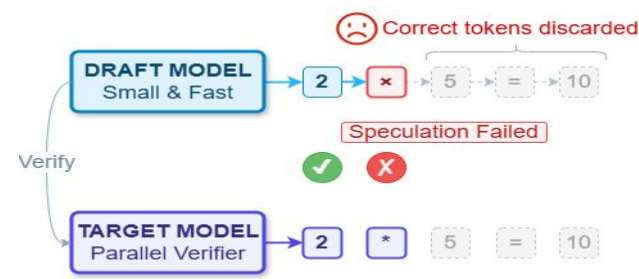
(1)强串行依赖：受自回归机制限制，Token 必须严格按序逐个生成，天然缺乏并行性，导致硬件利用率难以提升。

(2)算术强度低：单步生成需全量搬运数百GB 的模型权重，但仅服务于极少量的Token 计算，计算与访存开销严重失衡。

(3)显存带宽屏障：由于“搬运多、计算少”，GPU 核心长期处于严重的“数据饥饿”状态，推理速度上限被显存带宽死死锁死。

(4)投机解码加速范式：通过“草拟-验证”机制实现局部并行化，利用一次权重搬运验证多个Token，有效均摊访存开销，突破带宽限制。

核心问题



(a) Standard Speculative Decoding

验证机制过于僵化

- 依赖逐词精确匹配，缺乏语义包容性。
- 强能力草拟模型与僵化验证规则不匹配。
- 微小词法差异导致冗余重算，侵蚀加速收益。

亟需一种“语义感知”且“模式自适应”的轻量化校准验证框架。

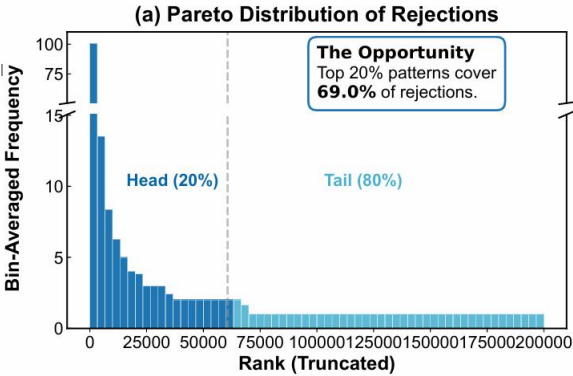
CSD:通过语义校准打破投机解码“僵化验证”

[ACL 2026 Main

洞察：拒绝模式中的规律与挑战

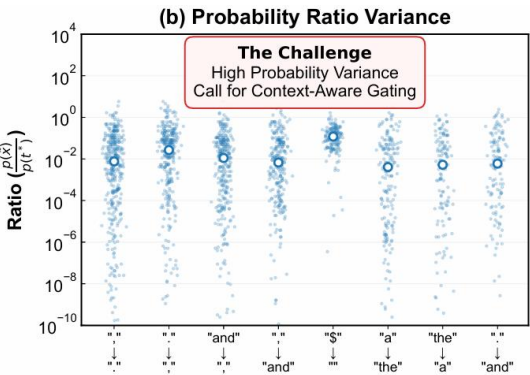
关键洞察 1：

拒绝模式呈现**长尾分布**，前 20% 的头部拒绝模式覆盖 69.0% 的总拒绝量。
通过轻量化修正记忆**捕捉高频规律**，即可挽救多数失败投机。



关键洞察 2：

相同词对在不同语境下的置信度**跨越数个数量级**。
语义有效性**强依赖上下文**，需引入动态门控区分合法变体与事实错误。



单一维度的松弛难以平衡性能与安全

Variant	MATH500				HumanEval			
	Acc	AR	Tp	Spd	Acc	AR	Tp	Spd
SpecDecode (Baseline)	45.4	63.6%	28.8	1.00×	76.8	59.7%	30.1	1.00×
SD w/ OCM	37.8	83.1%	36.6	1.27×	70.7	71.6%	37.2	1.24×
SD w/ SCG	43.6	88.7%	38.2	1.33×	70.7	88.6%	44.4	1.48×
CSD (Ours)	48.0	79.6%	35.5	1.23×	79.3	67.9%	37.0	1.23×

关键洞察 3：

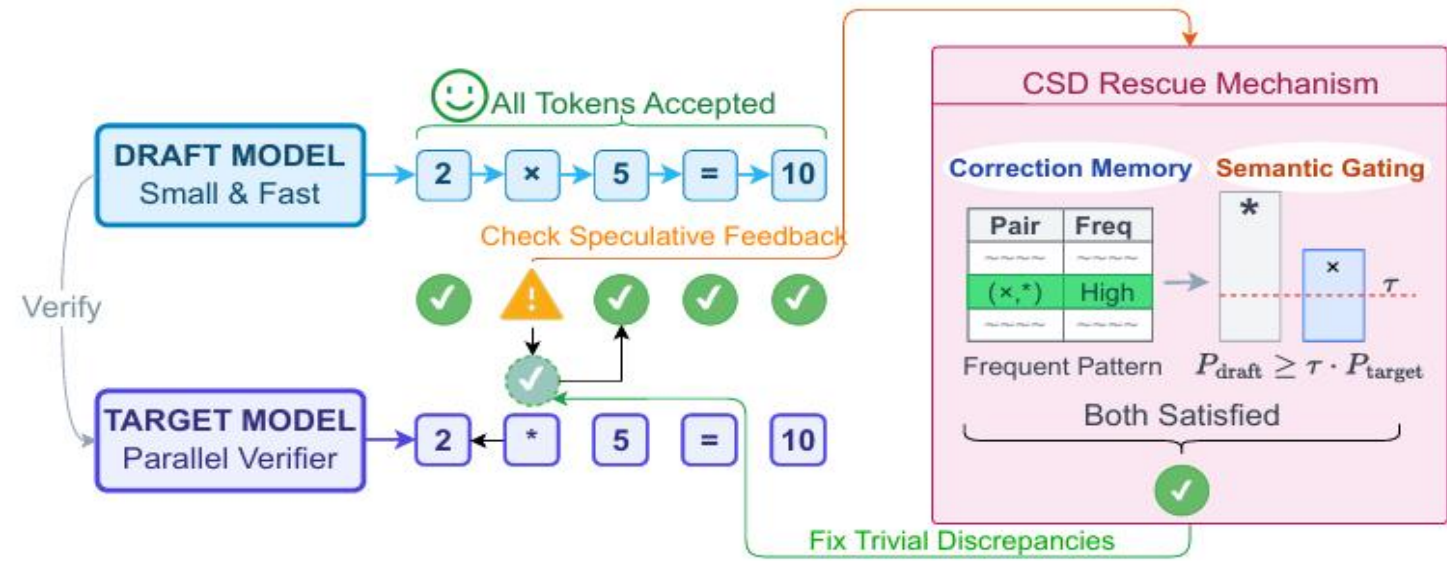
- 单纯频率引导的缺陷：历史规律是**语境无关**的。若无语义约束，高频误拒的修复会演变为严重的**逻辑错误**。
- 单纯语义门控的局限：缺乏模式先验的约束，单纯的概率松弛会导致验证**过度宽松**，导致精度**显著退化**。
- 设计共识：必须通过“**频率引导选择**”与“**概率守护接受**”的双重标定，才能在**严格保持精度**的前提下获得加速收益。

CSD 框架设计

[ACL 2026 Main

核心目标

- 1.面向大语言模型推理，在**严格精度保持**的前提下突破加速瓶颈。
- 2.通过“语义校准”机制，挽回**被刚性规则误拒**的有效 Draft Token，最大化投机收益。



实验结果

[ACL 2026 Main

实验结果

实验结果：突破加速瓶颈

Method	GSM8K (8-shot, Pass@1)			MATH500 (4-shot, Pass@1)			HumanEval (0-shot, Pass@1)			CNN/DM (0-shot, ROUGE-L)			Avg. Spd
	Acc	Tp	Spd	Acc	Tp	Spd	Acc	Tp	Spd	Acc	Tp	Spd	
Llama-3 Series													
Vanilla Decoding	92.6	13.4	1.00×	46.0	15.2	1.00×	76.8	15.9	1.00×	20.3	15.3	1.00×	1.00×
SpecDecode	92.3	23.3	1.74×	45.4	28.8	1.89×	76.8	30.1	1.90×	20.3	22.8	1.49×	1.75×
Lossy SD ($\tau = 0.6$)	92.3	24.2	1.80×	49.4	30.6	2.00×	78.0	32.3	2.04×	20.3	23.6	1.55×	1.85×
SWIFT	92.6	12.9	0.96×	46.4	16.5	1.09×	76.8	17.5	1.10×	20.2	17.5	1.14×	1.07×
Lookahead	92.6	11.1	0.83×	46.6	12.5	0.82×	76.8	14.4	0.91×	20.3	12.5	0.82×	0.84×
CSD (Ours)	92.6	24.5	1.83×	48.0	35.5	2.33×	79.3	37.0	2.33×	20.3	24.4	1.59×	2.02×
Qwen-2.5 Series													
Vanilla Decoding	91.7	13.9	1.00×	82.2	15.4	1.00×	88.4	14.9	1.00×	19.9	15.2	1.00×	1.00×
SpecDecode	91.5	22.2	1.59×	81.4	29.6	1.92×	89.0	26.9	1.81×	19.9	20.2	1.33×	1.66×
CSD (Ours)	92.2	25.2	1.81×	83.2	32.8	2.12×	88.4	29.0	1.95×	19.9	23.7	1.57×	1.86×

1. 突破理论加速瓶颈

在 Llama-3 (70B/1B) 与 Qwen-2.5 (72B/7B) 上全面超越现有方案，峰值加速比达到 2.33x。

2.. 严格无损乃至“反向涨点”

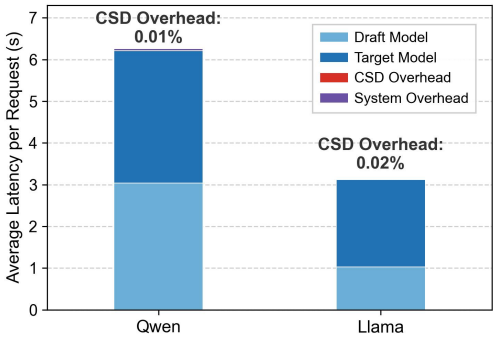
不仅在所有任务上守住了精度底线，更在复杂推理任务上实现反超（HumanEval 精度 +2.5，MATH500 精度 +2.0），证明找回的 Token 确实是有价值的有效轨迹。

方案泛化性

Task	Acc		AR		Speedup	
	Spec.	Univ.	Spec.	Univ.	Spec.	Univ.
HumanEval	79.2	78.0	67.9%	71.1%	2.33×	2.45×
Math500	48.0	50.0	79.6%	74.3%	2.32×	2.23×
GSM8K	92.6	92.8	76.4%	74.5%	1.83×	1.80×
CNN/DM	20.3	20.3	55.3%	54.4%	1.59×	1.59×
Avg.	60.0	60.3	69.8%	68.6%	2.02×	2.02×

仅需在通用语料上进行单次校准，即可在全任务上实现与任务特化相媲美的无损加速，展现出极强的领域无关泛化能力。

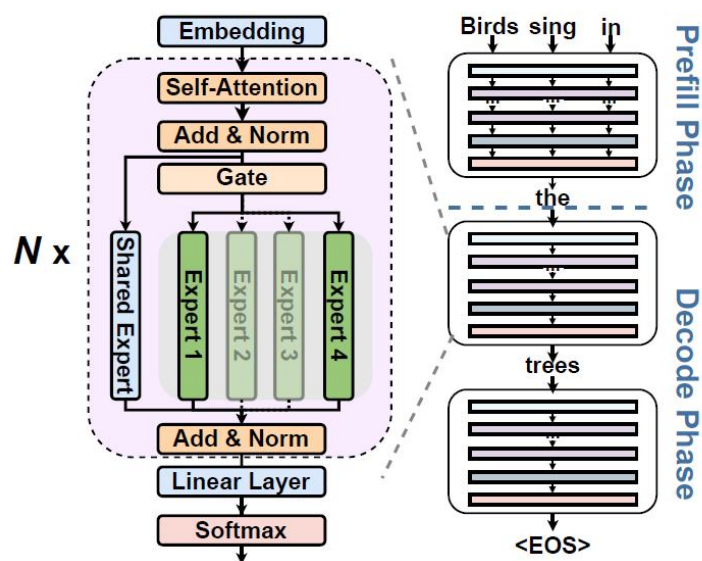
极低开销



CSD 的额外开销仅占端到端推理总延迟的 0.01% - 0.02%，以几乎为零的代价，换取了巨大的加速收益。

核心背景

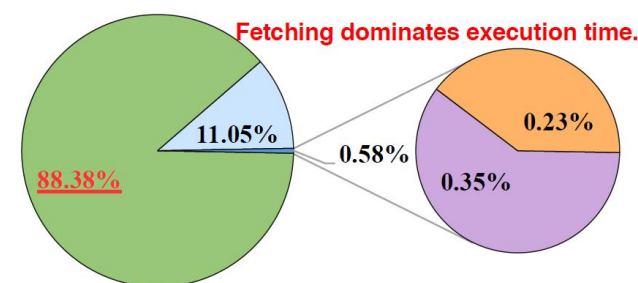
MoE 模型通过稀疏激活专家子集，在保持精度的同时降低计算量，已成为大语言模型高效部署的关键架构



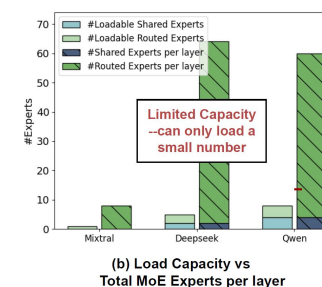
关键问题：专家参数规模庞大，导致内存访问成为推理瓶颈

核心挑战

1.数据移动主导延迟：专家读取占总执行周期的~**88%**，计算仅占12%



2.内存容量限制：片上缓存仅能存储少量专家

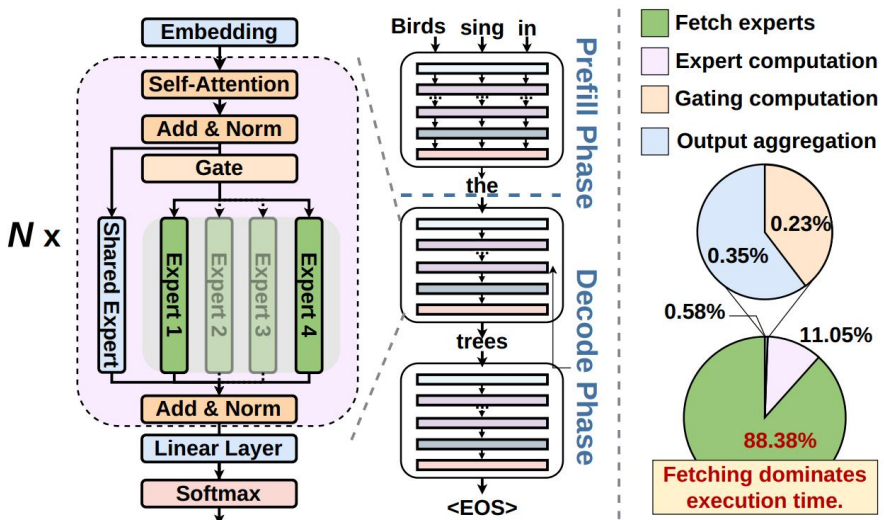


3.预取效率低：传统静态预取无法适配动态专家访问模式，导致带宽浪费和流水线阻塞

EARTH: 基于信息熵表征的MoE编码加速

[ASPLOS 2026]

核心问题



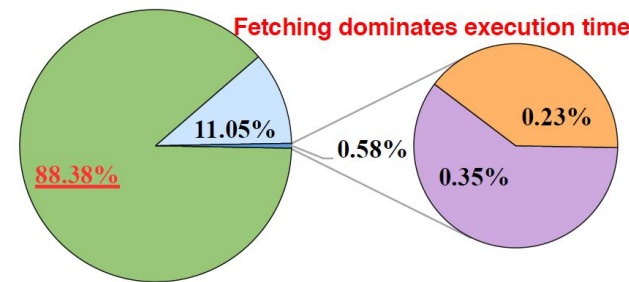
MoE 推理中，专家的部署成为了新的瓶颈 (专家存取占比 88%，为主要延迟来源)

- 访存开销大
- 专家选取不规则

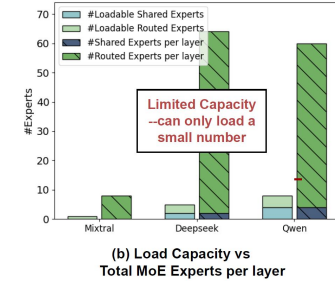
需要一个有效降低访存需求，高效且命中率高的优化方案

核心挑战

1.数据移动主导延迟：专家读取占总执行周期的 ~88%，计算仅占 12%



2.内存容量限制：片上缓存仅能存储少量专家



3.预取效率低：传统静态预取无法适配动态专家访问模式，导致带宽浪费和流水线阻塞

EARTH: 基于信息熵表征的MoE编码加速

[ASPLOS 2026]

核心思路

通过“双熵编码 + delta 感知预取 + 模式复用”的硬件 - 软件协同设计，在保持精度的前提下，降低内存流量并提升预取效率

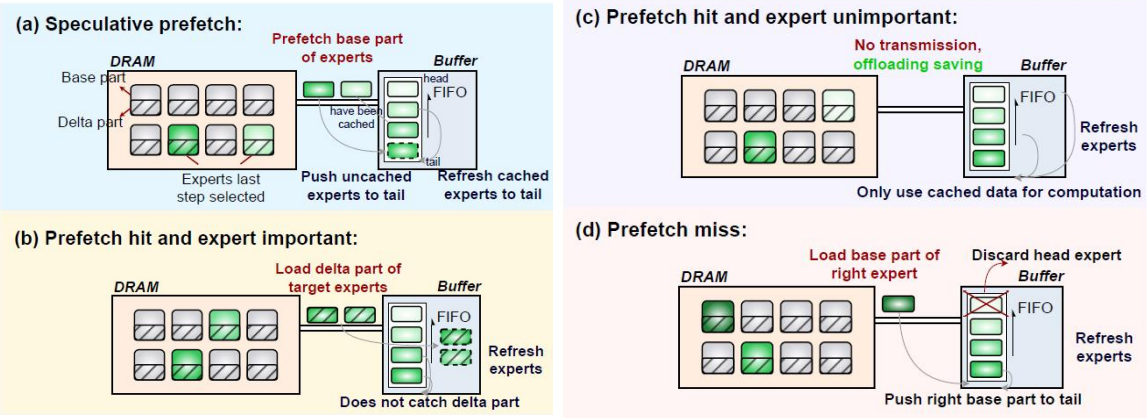
三大创新

1. 双熵编码（Dual-Entropy Encoding）：将专家参数分解为高信息基部分（Base）和增量部分（Delta），实现紧凑存储。
2. Delta 感知投机预取：预加载预测专家的基部分，根据专家重要性动态决定是否加载增量部分。
3. 模式复用：利用基 - 增量对的高冗余性，通过查找表（LUT）复用重复模式，减少冗余传输与计算。

技术细节——双熵编码与预取策略

双熵编码：

- 基于权重比特熵分布
- 无需复杂解码开销，适配动态精度需求



Delta 感知预取：

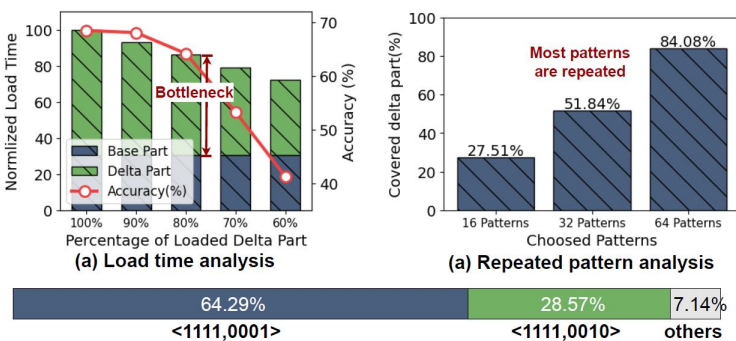
- 重要专家：加载 Delta 部分，全精度计算
- 非重要专家：直接使用Base部分，节省带宽
- 未命中：加载目标专家Base部分，最小化延迟损失

EARTH: 基于信息熵表征的MoE编码加速

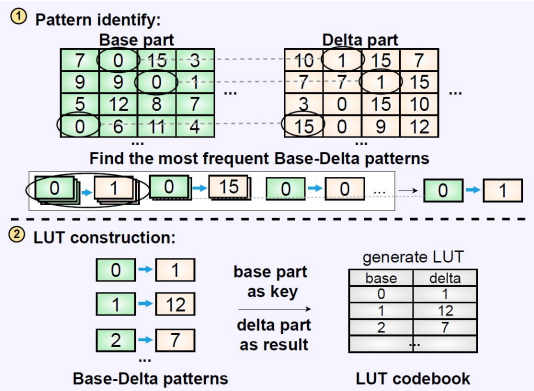
[ASPLOS 2026]

技术细节——模式复用

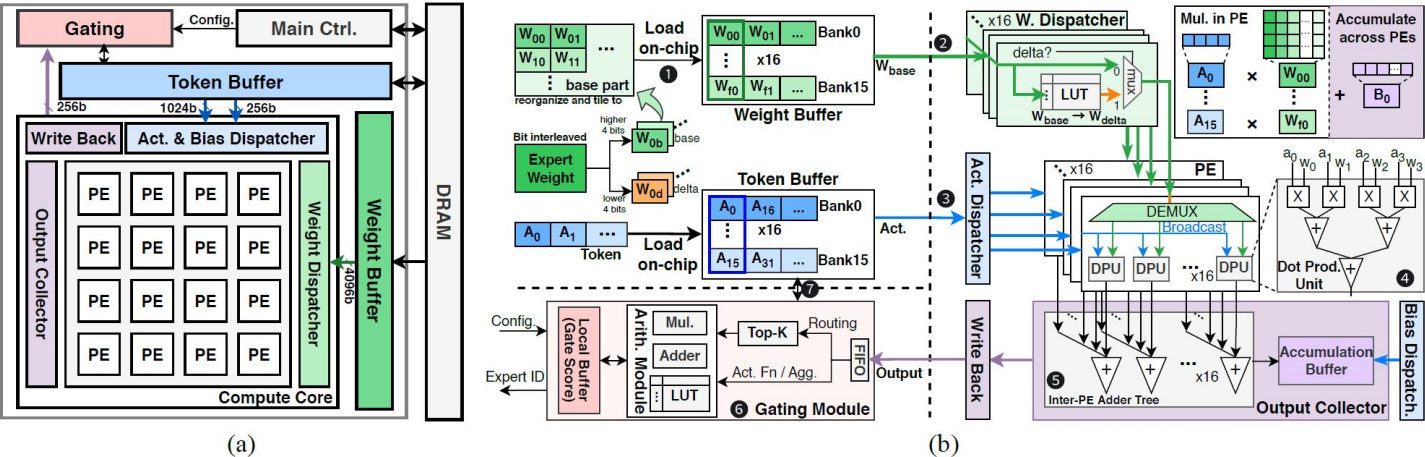
核心发现：
Base-delta对
存在高重复率



结合硬件优化：
离线构建 LUT，存储
高频Base-delta映射；
运行时通过Base部分
查询 LUT，复用
Delta 值



硬件架构



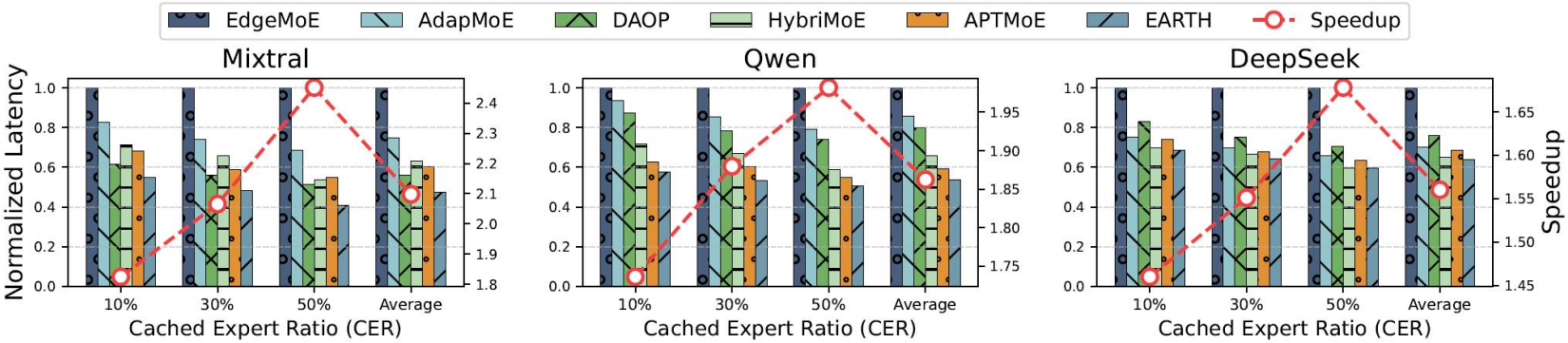
- 将数据分块存储Base部分与Delta 部分，支持细粒度并行访问
- 通过协调预取、解码与计算，实现数据传输与计算重叠

EARTH: 基于信息熵表征的MoE编码加速

[ASPLOS 2026]

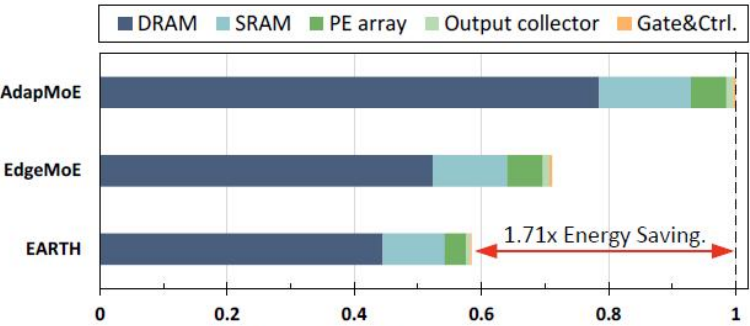
性能&精度

相较于主流Baseline (EdgeMoE、AdapMoE等)，实现 **1.56x-2.10x** 端到端加速，同时精度维持良好。DeepSeek 中，保留80% 重要专家时，Rouge-L 精度仅下降0.40%。

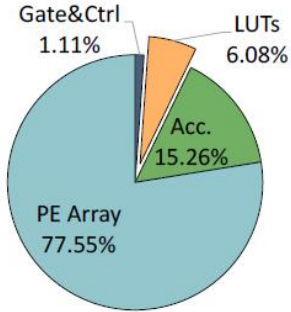


能效&面积

- EARTH减少外存访问量，优化数据流向，降低计算冗余
- 相较于 AdapMoE 和EdgeMoE，实现至多**1.71x** 能耗节省。



(a) Normalized Energy Breakdown.



(b) EARTH Area Breakdown.

- 总面积：**27.52mm²**
- LUT 与控制逻辑仅占 **6.08%**，PE 阵列（核心计算单元）占 **77.55%**，硬件复杂度低

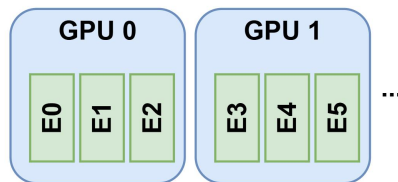
STEP: 基于时空预取的高效 MoE 推理优化

[ISCA 2026]

现有方法的不足

(1) 专家并行

分布式存储专家摊销访存需求
但硬件要求高，通信开销大。

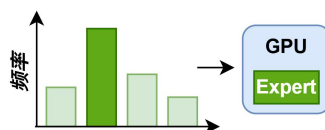


(2) 量化/剪枝

减少推理计算量
但重要专家不断变化且需要高精度/保留，动态
量化/剪枝开销大，实际收益低

(3) 静态预取

固定规则预取专家，掩盖传输延迟
但专家选取不规则，命中率低



(4) 动态预取

基于序列预测专家激活，预取命中率更高
但预测开销大且不稳定

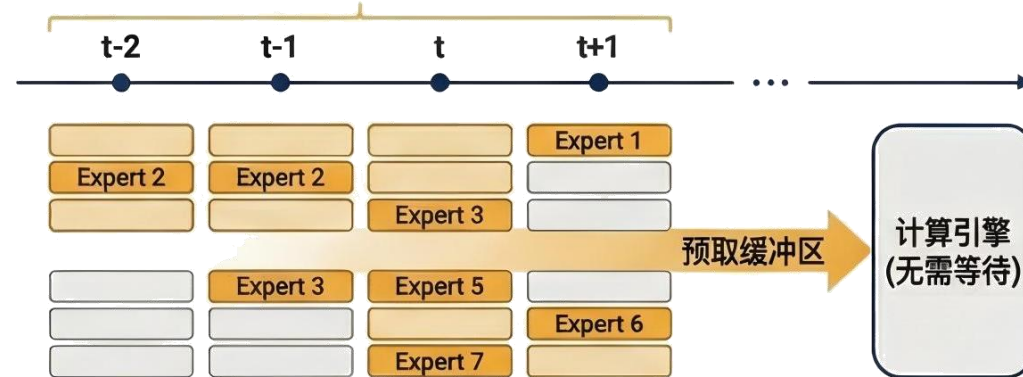
STEP 分支：预取策略（在哪取）

时空局部性利用 (t-2 至 t+1)

观察到Token选取的重复模式，最近使用的专家在后续几步中具有高重用概率。

自适应预取窗口

根据任务和层级的预测准确率，动态调整预取范围，以隐藏PCIe传输延迟。



预取缓冲区高效加载

提前将预测的高频专家（黄色高亮）载入显存，
确保计算引擎无需等待数据。

完整的 MoE 效率环路

STEP: 基于时空预取的高效 MoE 推理优化

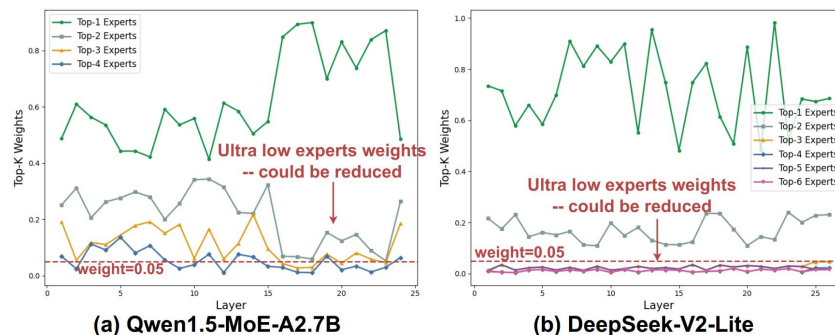
[ISCA 2026]

关键洞察：MoE 专家选取蕴含优化机会

洞察 1:

专家权重存在**空间不均衡**现象，部分专家对结果贡献低，但仍然需要加载

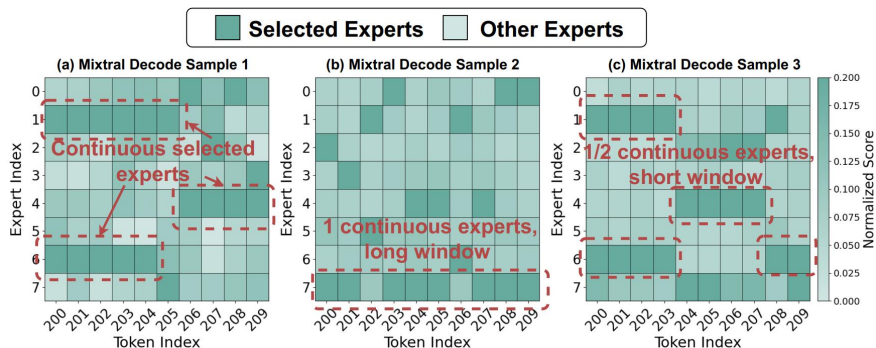
可以减少部分低贡献专家加载，减少访存



洞察 2:

专家权重存在**时间局部性**特征，相邻 token 会倾向于重复使用相同专家

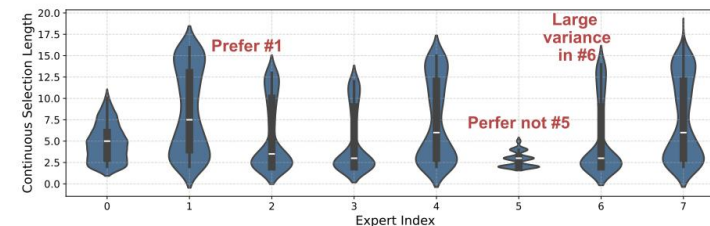
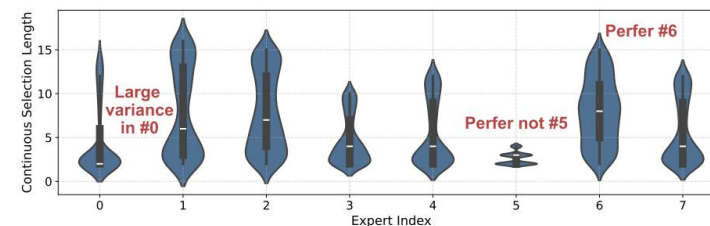
可以利用局部性预测选取专家



洞察 3:

专家权重倾向存在**动态变化**，根据 token/layer/task 不同，专家存在不同的倾向性

需要自适应预取机制



STEP: 基于时空预取的高效 MoE 推理优化

核心目标

- 利用空间和时间局部性解决 MoE 推理现有的高卸载延迟和低专家重用率问题
- 离线专家修剪配合在线专家预取，最大化减少延迟

关键设计

1. 离线利用空间局部性进行专家数量修剪

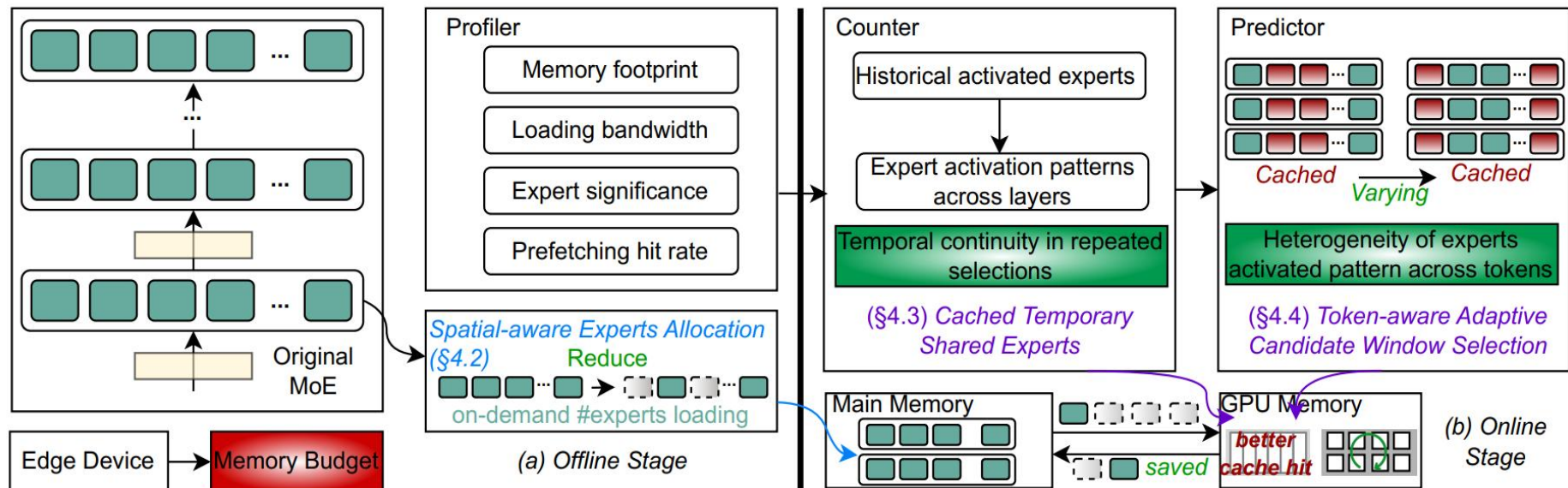
基于权重阈值，减少 top-k 选取专家数量 → 降低计算量，减少内存访问需求

2. 在线基于时间连续性实现重要专家预取

识别高频选取专家，并将其转换为临时共享专家，提前并固定加载到 GPU → 提高内存效率，减少加载延迟

3. 词元感知的自适应窗口机制

在选取专家时采用动态预取窗口，基于预测准确率调整大小 → 提升预取准确率，平衡预取收益与开销



STEP: 基于时空预取的高效 MoE 推理优化

[ISCA 2026]

预取机制

临时专家预取:

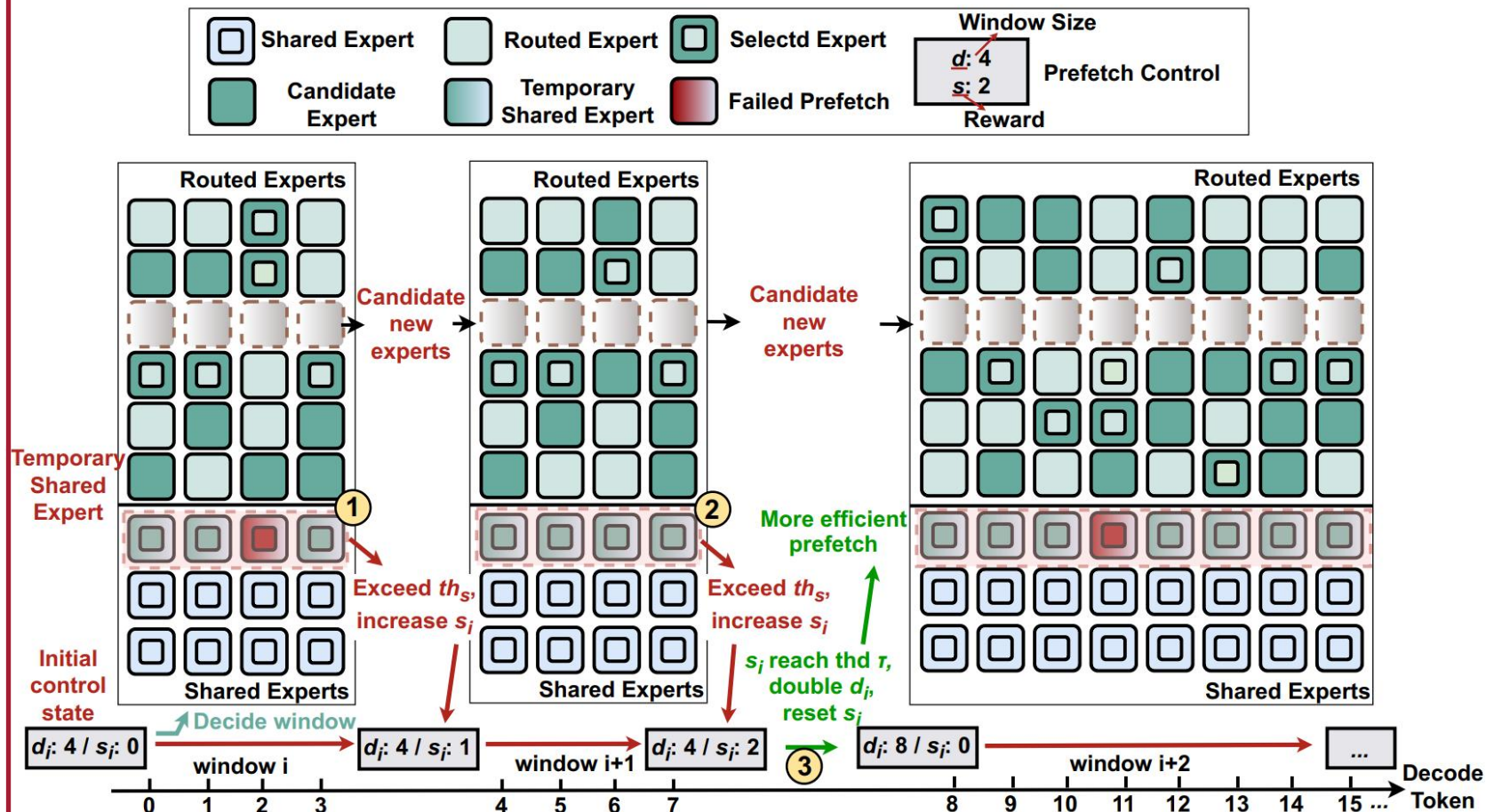
- 统计上一个窗口的历史专家选取情况
- 投票选出最高频的专家
- 将其转为临时共享专家，在下一个窗口始终缓存

有效减少重复专家加载

预取窗口动态调整:

- 统计本窗口预取专家命中率，是否缓存到被选取专家
- 如果命中率高 $\rightarrow$ 扩大窗口
- 如果命中率低 $\rightarrow$ 缩小窗口

显著提高内存利用率，降低预取开销

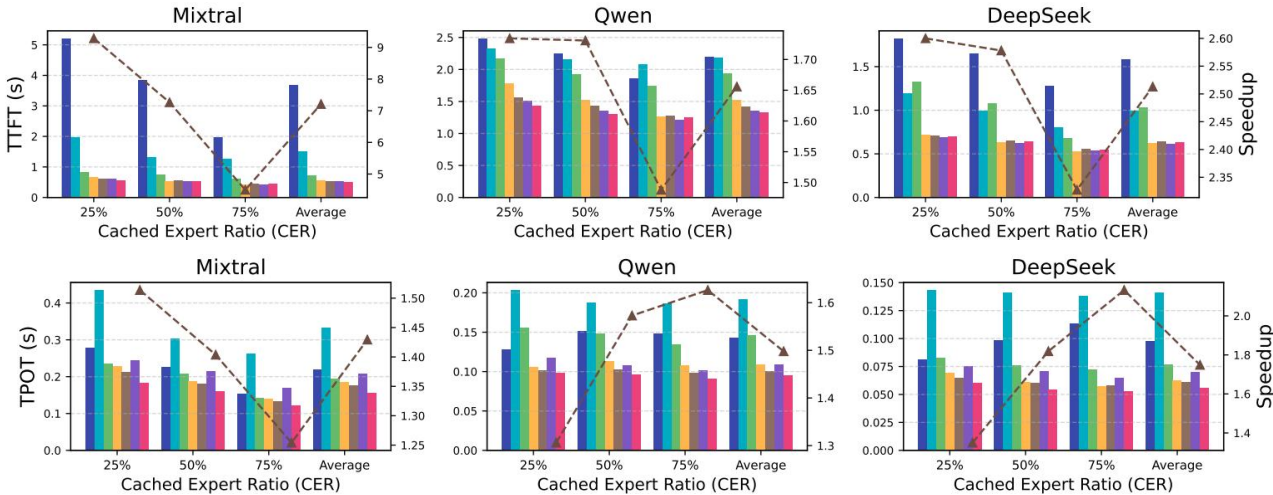


STEP: 基于时空预取的高效 MoE 推理优化

[ISCA 2026]

实验结果

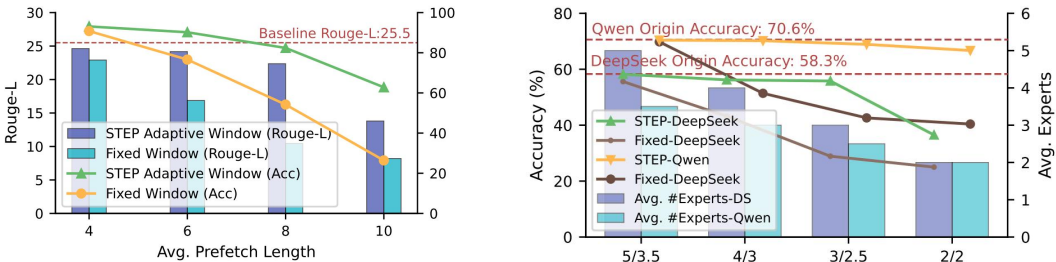
性能提升



相比现有的SOTA方案，能实现：
Prefill：最高 **3.12×** 加速
Decode：最高 **2.22×** 加速
在平均减少1-2个专家的情况下，**精度几乎无损**
专家预取成功率极高，超过 **90%** 预取命中率

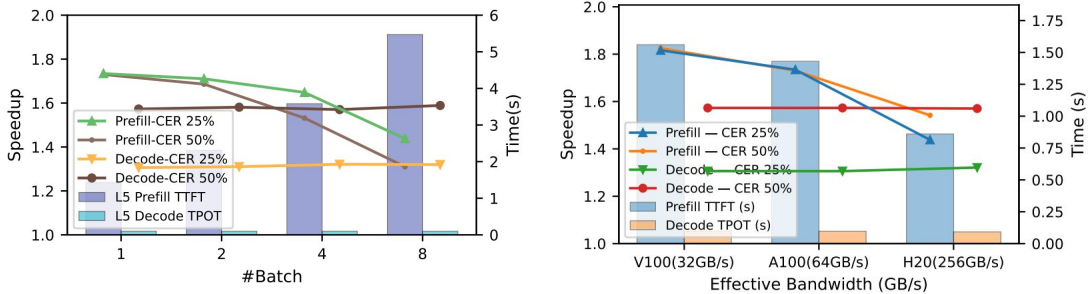
Method	Avg. #Experts	Window Size	MMLU				CNN/DM		LongBench	
			Accuracy (%)				Rouge-L	Hit Rate	Rouge-L	Hit Rate
Origin	2	—	77.3	75.8	64.2	71.3	35.6	—	34.3	—
	1.75	6 8	77.0	75.4	64.1	70.2	35.4	98.8 94.3	34.1	95.6 90.7

组件有效性



提出的动态窗口机制和自适应离线修剪机制实现了更优的效果

方案扩展性



不同的 batch size 和 带宽设置下，STEP 都能实现有效加速

极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

🕒 8月 📍 深圳 🧑 1000人

AiCon

全球人工智能开发与应用大会

会议时间：8月21-22日

- Agentic AI
- 轻量化与高效推理
- 多模态应用
- AI + IoT 场景实践
- AI 工业化落地

🕒 10月 📍 上海 🧑 1200人

QCon

全球软件开发大会

会议时间：10月22-24日

- AI Agent
- Vibe Coding
- 智能可观测
- 推理基建
- 模型攻防
- AI x 创造力

🕒 12月 📍 北京 🧑 1000人

AiCon

全球人工智能开发与应用大会

会议时间：12月18-19日

- 大模型架构创新
- 多模态 AI 产业融合
- 具身智能
- AI for Science
- 大模型安全

AiCon

全球人工智能开发与应用大会

深圳站

探索 AI 应用边界

Explore the limits of AI applications

2026 年 8 月 21-22 日 / 深圳湾万丽酒店



参会咨询

专题: Harness Engineering: 模型之外的智能体工程

专题出品人: 柴思远

智谱 / 副总裁



专题: 超级个体与蜂群智能的共生进化

专题出品人: 张昊阳

EvoMap / 创始人



专题: AI Agent 高价值商业场景实战

专题出品人: 李响 博士

平安科技 / 智能两核 AI 部总经理



专题: 超级 App 的 AI 原生化: 端智能、端云协同与用户体验重构

专题出品人: 黎三平

京东 / 端技术负责人



专题: AI 原生新范式: Coding Agent 重构软件研发全流程

专题出品人: 林香鑫

网易游戏 / 高级技术经理



专题: 智能体时代的数据供给与治理

专题出品人: 陈廷梁 (王赟)

数新智能 / 创始人兼 CEO



协同之效

图编排与高利用率的算子重构

“解决“用不好/不匹配”的问题。

实现软件调度与底层硬件核心（NPU/Tensor Core）的有效兼容。”

BEEMS: 计算图优化实现内存“削峰填谷”

[PPOPP'26]

视觉模型的部署压力

	ResNet-50	MobileNetV3-Small	YOLOv7-Tiny	Mask R-CNN	ViT-Base	CLIP	BLIP-Large	Stable Diffusion
参数量	25.6M	2.54M	6.24M	~44M	86.6M	~86M	~400M	~860M
模型大小 (FP32)	97.8MB	9.71MB	23.8MB	~170MB	~350MB	285.4MB	852.9MB	—
推理显存需求 (BS=1)	~0.5GB	~40MB				~0.3–0.4GB	~1GB	10GB(1080p)

减少内存的同时，
控制推理耗时。

- 云端（GPU集群）
- 优势：吞吐量为核心
- 趋势：



- 模型内存需求下降 → 吞吐量提升
- 同时延迟增加 → 吞吐量受损

- 边缘 / 嵌入式
- 资源极度有限
- 趋势：



- 模型内存需求下降 → 成本压缩
- 同时延迟增加 → 用户体验下降

BEEMS: 计算图优化实现内存“削峰填谷”

[PPOPP'26]

现有计算图优化技术

(1) 子图替换

替换等价子图、融合算子，提高吞吐；
但通常会 **增加内存**（因更大的 fused op）。

(2) 算子切分

大算子按维度拆成多段；
降低单次内存，但降低并行度 → **延迟变高**。

(3) 张量持久化

将不活跃张量驻留在内存中，
节省内存但增加数据传输开销。

(4) 重计算

不存中间结果；用重复计算换内存；
适用于有多个下游的节点；
减内存但增加算力消耗。

核心问题

Optimization	Mem.	Lat.	Target Tensors
Subgraph Substitutions	▲	▼	Activate Tensors
Operator Partitioning	▼	▲	Activate Tensors
Swapping	▼	▲	Inactivate Tensors
Rematerialization	▼	▲	Inactivate Tensors

优化技术之间“互相冲突”

- 内存优化通常让延迟变差

需要一个“内存感知、图区域感知”的统一优化框架

- 图结构复杂化
- 冗余计算
- 优化彼此抵消

任何优化策略都无法同时
进行**内存 & 延迟**的
双指标优化

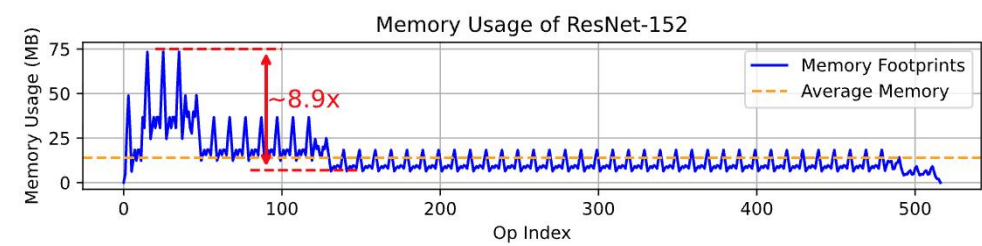
BEEMS: 计算图优化实现内存“削峰填谷”

[PPOPP'26]

洞察：内存模式提供了优化机会

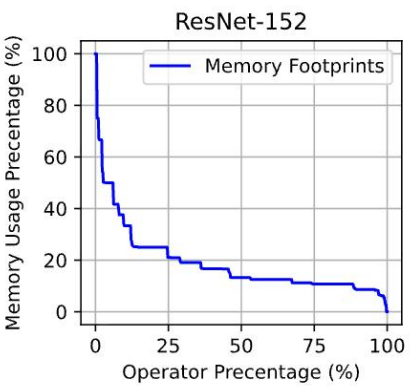
关键洞察 1：

NN推理时**峰值与平均内存**存在 $4.3\times-10.6\times$ 的巨大差距，可在低利用率节点上安全地做子图替换等优化，而不触碰峰值内存限制。



关键洞察 2：

高内存占用算子呈现**长尾分布**，对其进行内存削减技术（如**算子切分/交换**），**性能影响小**



naïve的技术叠加会带来劣化

AI Compiler	Subgraph Substitutions	Operator Partitioning	Swapping	Rematerialization	Conv Kernel Substitution
TVM [5]	✓	✗	✗	✗	✗
DTR [15]	✗	✗	✗	✓	✗
DNNFusion [17]	✓	✗	✗	✗	✗
TASO [13]	✓	✗	✗	✗	✗
MAGIS [4]	✗	✓	✓	✓	✗
MoteNN [3]	✗	✓	✗	✗	✗
G10 [32]	✗	✗	✓	✗	✗
BEEMS(Ours)	✓	✓	✓	✓	✓

关键洞察 3：

基于区域的隔离可避免不同优化策略（如子图替换与算子切分）之间相互干扰：子图替换、算子切分可独立执行，实现高效且无冲突的优化效果。

BEEMS: 计算图优化实现内存“削峰填谷”

[PPOPP'26]

核心目标

1. 面向视觉基础模型 (VFMs)，同时优化内存峰值与推理延迟。
2. 通过内存“削峰填谷”在计算图中对高/低内存节点做差异化优化

关键设计

峰值区:

内存紧张 → 优先削减内存

谷值区:

有内存富余 → 优先提升性能

区域感知: 避免互相冲突

(如子图替换扩大算子后又被切分导致额外 overhead)

四大组件

1. 内存分析

模拟张量生命周期，生成内存时序曲线
自动定位 峰值 与 谷值区段

2. 联合优化器

基于 profiling 选择合适策略
图结构去重避免重复搜索
迭代生成更优的内存/性能版本

3. 峰值削减

Kernel 替换 (Winograd / GEMM / FFT)
算子切分: 降低单次内存
Swapping 迁出不活跃张量
Rematerialization 以计算换内存

4. 谷值填充

子图替换 (基于 TASO)
利用富余内存提升性能
不增加整体峰值内存

BEEMS: 计算图优化实现内存“削峰填谷”

实验结果

[PPOPP'26]

内存/延迟约束下的优化

在 $\leq 105\%$ 原始延迟 下:

BEEMS 可减少至原来的 **15%–40%** 内存

在 $\leq 80\%$ 原始内存 的约束下:

BEEMS 延迟仅增加 **0%–10%** (部分模

在双目标优化中, BEEMS可以大幅缩减内存, 并控制延迟没有劣化 (甚至优化) 。

内存平滑效果

Peak-to-average 从 **4–10×** 降到约 **1.3×**

模型扩展性

模型加深, 内存模式改变, 优化幅度略下降。

大模型远端缓存调度：问题与挑战

背景

大模型内存压力持续升级：

- 推理：KV Cache 随序列长度线性增长
- 训练：激活值 + Optimizer State → TB 级需求
- 计算成本 < 数据搬运成本

本质：内存容量与带宽成为主要瓶颈

现有方法局限

传统 Offload / ZeRO / Recompute：

- 发生在 runtime，是被动的
- 缺乏全局视野，难以隐藏通信
- CPU 介入引入 bubble

Runtime Prefetch 为何失败？

- LLaMA3-8B @ Ascend 910C
 - Runtime-driven prefetch:
 - 延迟 5.5s → 15s (3× slowdown)
- 原因：
 - CPU 频繁介入，破坏流水，产生 bubble

远端内存访问不应是 runtime 行为，而应成为计算图的一部分。

HyperOffload: 远端缓存访问算子化及编译调度

算子化远端缓存

引入三类新算子:

- Prefetch (远端 → 设备)
- Store (设备 → 远端)
- Detach (释放设备驻留)

缓存操作成为计算图的一部分。

执行顺序优化

同一 DAG, 不同执行顺序 → 不同内存峰值 & 不同 stall

- Prefetch 太早 → 显存暴涨
- Prefetch 太晚 → 计算阻塞
- 准时 Prefetch → 计算-通信重叠 (最优)

本质: 在图级别做全局调度优化

Runtime Offload	HyperOffload
运行时触发	编译期重写
局部决策	全局调度
易产生 bubble	可隐藏通信
CPU 介入	纯图级执行

HyperOffload: 系统实现与效果

系统实现

- 基于 MindSpore 图级优化
- JIT 自动重写
- 对用户透明
- 运行于 SuperNode 远端内存架构

训练场景优化效果

大幅降低了未掩盖的通信开销

推理场景优化效果

Metric	Baseline	Hierarchical Mem.	Δ
Defragmentation Events	57	0	Eliminated
Prefill Latency	129.33 s	99.41 s	-23.13%
End-to-End Latency	187.21 s	161.41 s	-13.78%
Peak Memory (MB)	59.8	44.0	-26.46%

推理耗时与内存均大幅降低



HYPEROFFLOAD: GRAPH-DRIVEN HIERARCHICAL MEMORY MANAGEMENT FOR LARGE LANGUAGE MODELS ON SUPERNODE ARCHITECTURES

Fangxin Liu^{1,*}, Qinghua Zhang^{2,*}, Hanjing Shen^{1,*}, Zhibo Liang²,
Li Jiang^{1,†}, Haibing Guan¹, Chong Bao², and Xuefeng Jin²

¹Shanghai Jiao Tong University, Shanghai, China
²Huawei Technologies Co., Ltd., China

ABSTRACT

The rapid evolution of Large Language Models (LLMs) towards long-context reasoning and sparse architectures has pushed memory requirements far beyond the capacity of individual device HBM. While emerging supernode architectures offer terabyte-scale shared memory pools via high-bandwidth interconnects, existing software stacks fail to exploit this hardware effectively. Current runtime-based offloading and swapping techniques operate with a local view, leading to reactive scheduling and exposed communication latency that stall the computation pipeline.

In this paper, we propose the SuperNode Memory Management Framework (**HyperOffload**). It employs a compiler-assisted approach that leverages graph-driven memory management to treat remote memory access as explicit operations in the computation graph, specifically designed for hierarchical SuperNode architectures. Unlike reactive runtime systems, SuperNode represents data movement using cache operators within the compiler’s Intermediate Representation (IR). This design enables a global, compile-time analysis of tensor lifetimes and execution dependencies. Leveraging this visibility, we develop a global execution-order refinement algorithm that statically schedules data transfers to hide remote memory latency behind compute-intensive regions. We implement SuperNode within the production deep learning framework MindSpore, adding a remote memory backend and specialized compiler passes. Evaluation on representative LLM workloads shows that SuperNode reduces peak device memory usage by up to 26% for inference while maintaining end-to-end performance. Our work demonstrates that integrating memory-augmented hardware into the compiler’s optimization framework is essential for scaling next-generation AI workloads.

昇思MindSpore 2.8版本正式发布，为超节点而生的HyperParallel架构，训推更灵活、更高效

开发者

发表于 2026/01/29

经过昇思MindSpore开源社区开发者们几个月的开发与贡献，现正式发布昇思MindSpore2.8版本。

面对模型规模持续增长、结构日趋不规则、异构性日益显著的三大紧迫挑战，昇思MindSpore通过与超节点协同设计，推出了HyperParallel架构。此架构将超节点视作一台超级计算机进行编程与调度，其核心由三大关键技术支撑：HyperShard极简声明式并行编程、HyperOffload多级智能卸载，以及HyperMPMD非规则异构并行。

在基础框架演进方面，提供了Dispatch功能、引入saved_tensors_hook机制和算子级注册机制增强动态图能力，并开放多种框架自定义能力提供更灵活、高效的框架扩展机制。

在大模型推理能力提升方面，与SGLang社区合作，提供MindSpore支持并适配Radix Cache等核心特性，升级适配vLLM v0.11.0最新版本并接入了昇腾ACLGraph图下沉功能。

2.HyperOffload实现全图自动的数据层次化管理，显著提升显存利用率与训练吞吐

在超大规模模型的训练与推理场景中，硬件加速器的存储容量与数据传输带宽已成为制约系统性能的关键瓶颈。针对这一挑战，昇思MindSpore2.8版本提出并实现了一套基于全图编排的层次化存储管理方案HyperOffload，旨在通过系统级的架构优化，提升异构计算环境下的资源利用率与执行效率。

HyperOffload的核心创新在于数据传输的算子化抽象与流水线并行，打破了传统计算与通信的界限，将跨层级存储的数据搬运操作抽象为图层面的算子，纳入统一的计算图编排体系。依托昇思MindSpore的即时编译（JIT）引擎，系统能够对静态计算图进行深度分析与优化，自动构建计算与通信的并行流水线。通过在计算任务执行期间预取后续数据，该机制有效地掩盖了高延迟的I/O开销，显著降低了计算单元的空闲等待时间。

针对不同开发模式的需求，HyperOffload提供了灵活的接入方式：

- 静态图模式：实现了高度的自动化。对于绝大多数应用场景，用户无需修改原有训练脚本，框架即可通过全图编译自动完成计算图的重构与内存优化。基准测试表明，在启用该特性后，模型训练吞吐量平均提升超过10%。
- 动态图模式：考虑到动态执行的复杂性与灵活性，HyperOffload开放了底层的存储管理接口。允许用户根据模型逻辑，通过手动调用接口的方式精确控制模型参数与中间数据的卸载与加载时机。这种细粒度的控制能力能够在动态图调试或特定算法实现中，针对性地优化显存占用，实现定制化的内存管理策略。

温馨提醒：HyperOffload当前处于实验性阶段。虽然初步验证表明该架构在提升显存效率与计算吞吐方面具有显著优势，但针对特定复杂网络结构的适配仍需进一步验证。未来，我们将持续优化内存调度算法与全图编译策略，致力于在更广泛的生产环境中提供稳定、高效的异构存储解决方案。

参考链接：https://www.mindspore.cn/docs/zh-CN/master/api_python/mindspore/mindspore.jit.html

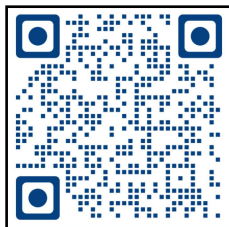


总结

- 大模型优化策略的架构设计原则：以最小（软硬）代价，最简单控制，利用稀疏范式
- 大模型存在更大的细粒度冗余/稀疏，且对存储与通信至关重要
- 稀疏性广泛存在于更为通用度计算场景

Homepage: <https://mxhx7199.github.io/>

Mail: liufangxin@sjtu.edu.cn



Home

落地场景



上海市公安局
gaj.sh.gov.cn



蚂蚁集团
ANT GROUP



阿里巴巴
Alibaba.com



[M]^s

HUAWEI MindSpore



IZ铸科技

ZTE中兴



上海交通大學

SHANGHAI JIAO TONG UNIVERSITY

感谢聆听

饮水思源 爱国荣校